

代 号 10701

学 号 0521121043

分类号 TP391.72

西安电子科技大学

硕士学位论文



题 (中、英文) 目 FPGA 结构和布局布线算法研究

Research on FPGA Architecture and

Placement Routing Algorithm

作 者 姓 名 赵刚 指导教师姓名、职务 段振华 教授

学 科 门 类 工学 学科、专业 计算机软件与理论

提交论文日期 二〇〇八年一月

创新性声明

本人声明所呈交的论文是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢中所罗列的内容以外，论文中不包含其它人已经发表或撰写过的研究成果；也不包含为获得西安电子科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中做了明确的说明并表示了谢意。

申请学位论文与资料若有不实之处，本人承担一切相关责任。

本人签名： 赵刚

日期 2008.03.10

关于论文使用授权的说明

本人完全了解西安电子科技大学有关保留和使用学位论文的规定，即：研究生在校攻读学位期间论文工作的知识产权单位属西安电子科技大学。本人保证毕业后离校后，发表论文或使用论文工作成果时署名单位仍然为西安电子科技大学。学校有权保留送交论文的复印件，允许查阅和借阅论文；学校可以公布论文的全部或部分内容，可以允许采用影印、缩印或其它复制手段保存论文。（保密的论文在解密后遵守此规定）

本人签名： 赵刚

日期 2008.03.10

导师签名： 

日期 2008.03.10

摘 要

论文设计了一种 FPGA 结构描述方法,解决了 FPGA 建模问题。FPGA 结构描述方法包含逻辑单元信息,互连线信息等 10 部分。当采用不同的 FPGA 芯片进行布局布线时,只需要使用结构描述方法重新定义这种 FPGA 芯片的结构,不需要改变布局布线工具。

为了配合 FPGA 编程下载,论文改进了划分网表算法,能够生成 LUT 配置信息文件。改进了布局布线算法,能够支持更多的商用 FPGA 结构特征,开发的布局布线工具在可布通性上和 VPR 接近,布局阶段能够减少 21%的逻辑单元交换次数,它在布局布线之后生成内部连接信息,布局信息和布线信息。这些信息提供给布局布线的下一阶段编程下载必要的支持,可以生成位流文件下载到 FPGA 中。

关键词: 现场可编程门阵列 布局 布线 验证

Abstract

An FPGA architecture description method is designed in the thesis to solve the problem of describing FPGA architecture. The method can be used to establish FPGA architecture model. Ten parts of information are contained in it, such as logic block information, wire segment information and so on. When using different FPGA chips to place and route, it only needs to use the method to redefine the FPGA chip architecture, without changing placement and routing tools.

The current academic partitioning netlist algorithm is improved to generate the LUT configuration information for FPGA programming and downloading. The placement and routing algorithm are also improved to support the commercial FPGA architecture. And a placement routing tool is developed to generate placement files, routing files and inner connecting files, which has the same routing capacity as that of VPR. The improved placement algorithm can decrease the times of swaping logic blocks by 23%. The four parts are the necessary information which can be used for FPGA chip programming and downloading.

Keywords: FPGA placement routing verification

目 录

第一章 绪论	1
1.1 引言	1
1.2 课题来源	1
1.3 研究背景和研究意义	2
1.4 论文章节安排	3
第二章 FPGA结构和相关术语定义	5
2.1 FPGA结构简介	5
2.1.1 逻辑模块	5
2.1.2 I/O模块	8
2.1.3 互连资源	8
2.2 FPGA结构相关术语定义	9
2.3 本章小结	12
第三章 结构描述式的FPGA建模方法	13
3.1 芯片总体信息	14
3.2 逻辑单元信息	15
3.3 I/O端口信息	18
3.4 布图信息	18
3.5 开关信息	20
3.6 互连线信息	20
3.7 通道信息	21
3.8 开关盒信息	23
3.9 面积模型	25
3.10 延时模型	25
3.11 本章小结	27
第四章 划分算法和布局布线算法的分析与研究	29
4.1 FPGA支持软件设计流程	29
4.2 FPGA划分和布局布线问题	30
4.2.1 划分算法回顾	30
4.2.2 FPGA布局问题	30
4.2.3 FPGA布线问题	31
4.3 VPACK和VPR的不足	33
4.4 改进的划分网表工具	34
4.4.1 改进的划分算法	34
4.4.2 一个简单的文件转换实验	35
4.5 改进的布局算法	38
4.5.1 模拟退火算法	38
4.5.2 改进布局算法流程	40
4.5.3 花费函数的确定	41
4.5.4 温度更新规则	43
4.5.5 搜索范围更新规则	43
4.5.6 逻辑单元交换规则	44
4.5.7 算法终止条件	44
4.6 改进的布线算法	44
4.6.1 采用布线资源图建立布线结构	44
4.6.2 改进的布线算法流程	44
4.6.3 查找从源点到目标点的路径	46
4.6.4 内部连线算法	47
4.7 实验方法与实验结果	49
4.7.1 实验方法	49

4.7.2 实验结果 49

4.8 本章小结 52

第五章 FPGA支持软件功能验证 55

5.1 FPGA软核设计 55

5.2 控制芯片设计 56

5.3 PC机程序设计 57

5.4 本章小结 57

第六章 结论与展望 59

6.1 结论 59

6.2 展望 59

致 谢 61

参考文献 63

作者在读期间研究成果 67

第一章 绪论

1.1 引言

1958 年全球第一块集成电路(IC)诞生, 结束了之前只能采用分立晶体管的历史。自 Gordon Moore 在 1965 年提出著名的摩尔定律以来, 集成电路的发展基本遵守这一定律。随着集成电路的不断发展, 1984 年, ALTERA 公司采用 CMOS 的 EPROM 工艺研制出首块可擦写的可编程逻辑器件(EPLD), 该芯片能够用紫外线擦除和重复编程。XILINX 公司于 1985 年推出了首块现场可编程门阵列 FPGA(Field Programmable Gate Array), FPGA 结合了可编程逻辑器件(PLD)的可编程性和掩膜可编程门阵列(MPGA)的通用连线结构, 具有较高的逻辑密度。通过对 FPGA 控制开关的编程, 可以很方便地把一个通用的 FPGA 芯片配置成为用户需要的数字电路。因此, 使用 FPGA 可以大大缩短电子产品的研发周期, 降低研发成本, 加快电子产品上市速度。

FPGA 器件产品的成熟, 促使其应用领域不断拓展, 在集成电路领域取得了越来越重要的地位和作用。现今 FPGA 的产品正向高密度, 低功耗, 集成片上系统, 内嵌模块等方向发展。通过嵌入处理器核能够取代 MCU 的一些应用, 在系统设计复杂度不断提高及新产品市场周期不断缩短的压力下, 把 FPGA 及微处理器的核心内嵌在同一芯片上, 构建可编程芯片系统 SOPC 成为最佳选择。针对 SOPC, XILINX 推出了 Micro Blaze 软核, ALTERA 推出了 Nios II 软核。

超大规模集成电路(VLSI)技术和计算机辅助设计(CAD)技术的共同发展使 FPGA 的技术日趋成熟并应用广泛。目前 XILINX, ALTERA 在 FPGA 市场处于领先地位, 分别有相应的支持软件工具(XILINX 的 ISE, ALTERA 的 Quartus II)来支持自己的产品。FPGA 支持软件工具一般包括电路输入, 逻辑综合, 工艺映射, 单元划分, 布局布线, 编程下载, 仿真等步骤。由于商用的 FPGA 芯片结构是保密的, 单元划分, 布局布线和编程下载都是由 FPGA 芯片供应商提供。其他步骤可以采用第三方公司的工具, 如 MENTOR 公司的 modelsim 仿真工具等。

1.2 课题来源

现场可编程门阵列 FPGA 器件的发展已经有二十多年的历史, 现在国际上的 FPGA 芯片主要由 XILINX, ALTERA, LATTICE 和 ACTEL 这 4 个公司生产。我国每年要购买大量的 FPGA 芯片用于系统开发, 主要用于消费电子, 工业控制, 航空航天等领域。由于我国目前没有自主知识产权的 FPGA 芯片和相应的支持软件工具, 在技术上受到一定限制, 且用于国防的军用高可靠 FPGA 芯片难以买到。

为了解决这一问题, 该项目主要是研究自主知识产权的 FPGA 芯片和相关的支持软件技术。

1.3 研究背景和研究意义

在商业方面, 目前使用的商用 FPGA 支持软件工具尚未公开其技术细节。在学术研究方面, 美国, 加拿大和日本等国家在 FPGA 支持软件技术方面做了很多的研究工作, 学术上居于领先地位。在 FPGA 支持软件流程中, 单元划分, 布局布线, 编程下载这 3 部分和具体 FPGA 器件有关, 在设计支持软件的过程中, 需要针对不同的芯片对该三个步骤进行研究设计。

单元划分要将工艺映射后的网表划分到实际的 FPGA 芯片逻辑单元, 把工艺映射后的网表中的 LUT, flip-flop 等合并, 组合成实际 FPGA 芯片逻辑单元, 生成逻辑单元网表。

布局布线是 FPGA 支持软件设计中关键的一步, 主要是确定逻辑单元的位置(布局), 确定逻辑单元之间的连线(布线)。布局布线的下一阶段是编程下载, 故布局布线结果需要提供给编程下载阶段作为输入。国外布局布线的学术研究主要是研究布局布线算法, 采用一种简单 FPGA 芯片结构来验证算法的优劣。由于布局和布线都是 NP 问题, 通常采用模拟退火算法来解决布局问题获得较高的解的质量, 采用拥塞协商布线算法来解决布线问题, 但是需要较长的运行时间。目前研究集中于降低功耗, 减少延时等, 没有考虑如何修改算法适用于某种实际的 FPGA 器件。

编程下载把布局布线的结果作为输入, 然后生成位流文件下载到芯片中。

为了开发具备我国自主知识产权的 FPGA 芯片和相应的支持软件工具, 需要考虑如何把布局布线算法应用到实际的 FPGA 芯片中。为了降低布局布线工具和 FPGA 芯片结构之间的耦合度, 需要一种描述方法对 FPGA 芯片建立模型, 并且布局布线工具能够提供编程下载所需要的布局布线结果, 如能完成这 2 项, 就可以实现对 FPGA 芯片的编程下载。

本文针对 FPGA 结构表示困难问题, 讨论了 FPGA 的结构特征, 定义了一套用于描述 FPGA 完整信息的数据结构类型, 这种结构描述方法能够表示 FPGA 芯片的具体细节, 当对不同的 FPGA 芯片布局布线时, 选择相应的结构描述文件, 不需修改布局布线工具。

另外为了能够生成编程下载所需要的布局布线结果, 开发了相应的划分工具和布局布线工具。划分工具改进了划分算法, 能够在把工艺映射后的网表划分成逻辑单元的同时生成 LUT 配置信息文件。布局布线工具支持结构描述文件定义的 FPGA 结构, 改进的模拟退火布局算法能够减少逻辑单元交换次数, 在没有降低布

局质量的情况下提高了布局算法的速度，并生成布局文件，在拥塞协商布线算法基础上增加了一个内部连线算法，能够生成布线文件和逻辑单元内部连线文件。这 4 个文件提供了编程下载必需的信息，最后可以生成位流文件用于编程下载。

支持软件中的主要步骤的算法最早都是由国外的学者提出的，目前技术相对较为成熟，而随着 FPGA 芯片技术的不断发展及在各方面的应用，FPGA 将会占芯片市场的主导地位，而高性能芯片和相关支持软件的设计在国内还是空白，因此设计研发 FPGA 芯片及支持软件对中国的芯片市场有着重要的意义。

1.4 论文章节安排

第一章，概述论文课题的来源，研究背景，研究意义，及论文的内容安排。

第二章，介绍FPGA结构和相关术语定义。

第三章，介绍设计的 FPGA 结构描述方法及如何对 FPGA 建立模型。

第四章，对开发的划分工具使用的算法及改进的布局布线算法进行了介绍，并与现有算法进行对比。

第五章，介绍了FPGA支持软件的功能验证方法，讨论如何对FPGA支持软件进行功能验证。

第六章，结论与展望。

第二章 FPGA 结构和相关术语定义

2.1 FPGA 结构简介

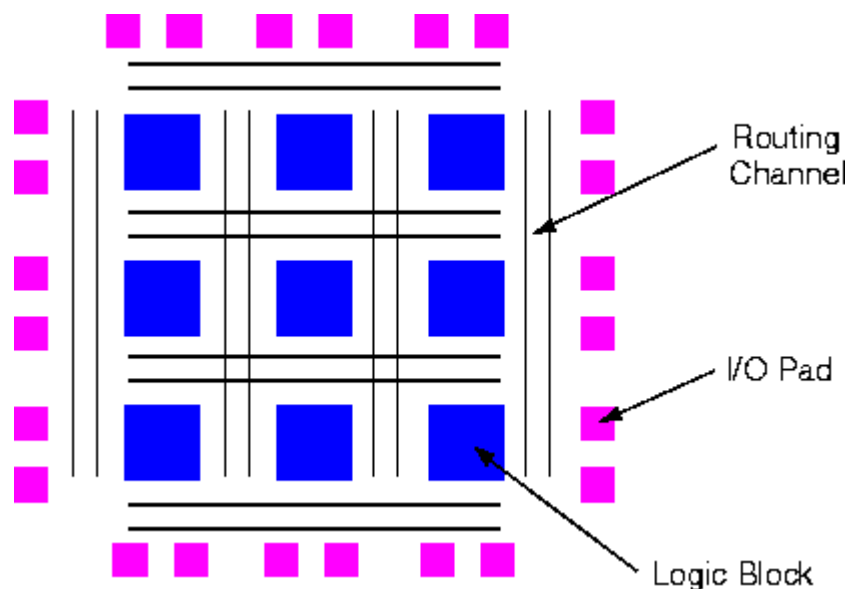


图 2.1 一种简单的 FPGA 结构

从图 2.1 中可以看出，FPGA 的结构^[1]主要是由逻辑模块，I/O 模块，互连资源组成。

2.1.1 逻辑模块

所有的可编程 ASIC 或者 FPGA 芯片都含有可以复制规则阵列的基本逻辑模块。共有 3 种逻辑模块：

- 1) 基于多路选择器的基本逻辑单元；
- 2) 基于查找表的基本逻辑单元；
- 3) 可编程阵列逻辑。

目前 SRAM 工艺，基于查找表技术的 FPGA 使用最广泛。本文讨论的都是这种类型的 FPGA。

一个简单的 2 输入查找表^[2]实现函数 $f = a + \neg b$ 如图 2.2 所示：查找表以多路选择器的形式描述，在这个查找表中，有 2 个地址线(a和b)和 2*2 个内存单元。查找表的输入是多路选择器的选择线，内存单元作为多路选择器的输入。例如：如果 $a=0$, $b=1$ 那么内存单元选择 01 连接到 f 的输出。内存单元的内容从等式 $f = a + \neg b$ 计算出来。

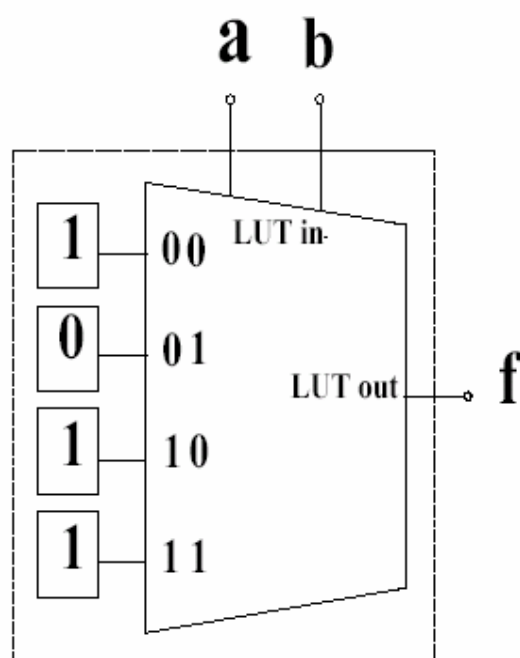


图 2.2 二输入的查找表实现逻辑函数 $f = a + \neg b$

输入端数为 n 的查找表是由 $2^n \times 1$ 个SRAM单元组成， n 个输入被译码以决定从哪一个SRAM单元读出信息送到唯一的输出端。在芯片初始化阶段，配置LUT中的信息位，决定LUT实现的函数功能。 n 输入的LUT可以实现任何 n 输入的函数，所以其功能度为2的 2^n 次方，4输入LUT的功能度即为256。但是 n 输入的LUT需要 2^n 个SRAM单元，每个SRAM位都至少由5个MOS管组成，这样对于4输入LUT就至少需要80个MOS管(未含译码电路)，而且用SRAM保存信息的FPGA在工作时需要外挂PROM芯片。不过，采用LUT作为基本模块对于CAD工具中的工艺映射实现比较有利，在工艺映射时只须考虑满足输入输出端的要求，而不必考虑模块内的具体逻辑。所以，基于LUT的逻辑模块仍然被广泛采用，例如，Xilinx公司XC2000/3000/4000/5000^[3]和Altera FLEX8000/10K系列^[4]。

对于LUT-based LM，主要是研究LUT的规模对芯片面积和性能的影响。目前的结论是：4输入单输出的LUT能获得最优的芯片面积，在LM中配备D触发器优于用组合逻辑构造D触发器，LM的粒度适当大些有利于布通率和时延的要求。这个结论为Xilinx和Altera大力发展基于LUT的FPGA提供了很好的理论基础，并为后来的研究者多次验证。J.He & J.Rose^[5]在此基础上，提出3-input LUT和数量4-input LUT的组合，可以节省15%的芯片面积，但是这样引入了布图的不规则性。事实上，Altera公司FLEX系列的结构中4输入的LUT是由两个3输入的LUT组合而成，在逻辑模块的某些配置模式中可以提供两个逻辑函数，提高了LM实现逻辑的灵活性。而Xilinx公司的XC4000结构^[2]中将两个4输入LUT和一个3输入的LUT组合在一起，使得逻辑模块可以方便地实现某些高输入端的模块。如图2.3：

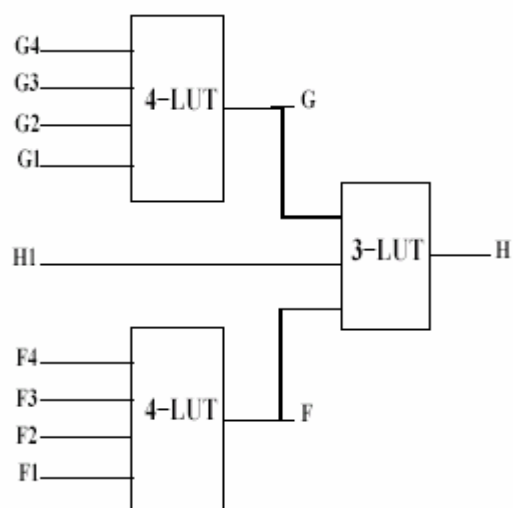


图2.3 XC 4000逻辑单元

基于查询表的逻辑模块是目前FPGA中使用最广泛的逻辑模块，一种基本的逻辑模块(Basic Logic Element)^[6]形式如图2.4，由四输入的LUT和一个D触发器构成，输出多路选择器MUX决定是组合还是时序输出(有时直接使用两个输出，这时不需要MUX)。

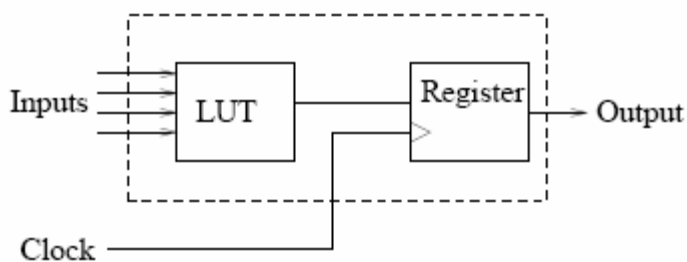


图2.4 基本逻辑模块(BLE)

图2.5是Altera FLEX 10k的逻辑模块^[4]，这是一种商业的FPGA逻辑单元的结构，每个单元内主要包含了一个4输入的查找表，和一个带同步使能的触发器，一个进位链，一个级连链。这种结构和基本的逻辑单元的结构相似，只是多了一些控制信号。

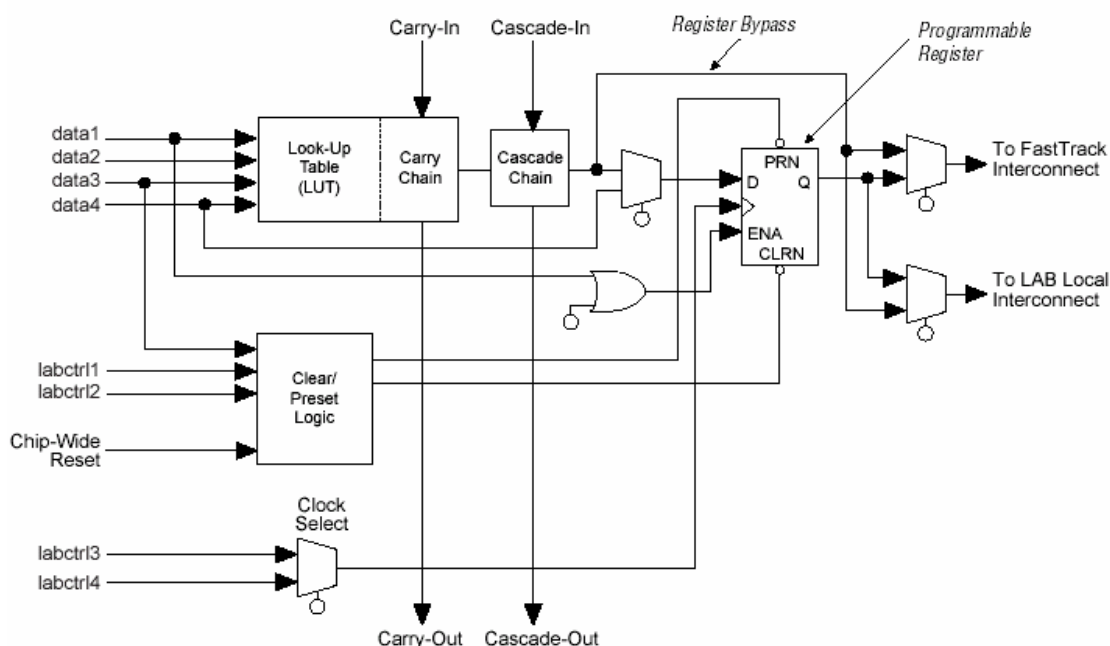


图2.5 FLEX 10k 逻辑模块

2.1.2 I/O 模块

一般一个 I/O 单元包含一个双向 I/O 缓冲器和一个寄存器，可以作为外部数据的输入寄存器，或者是内部数据的输出寄存器。有些 I/O 单元还要处理驱动片外逻辑信号，接收和调节外部信号以及处理诸如静电保护等。

如图 2.6^[4]是 FLEX10k 系列的 I/O 模块双向寄存器图。每个 IOE 从外围控制总线选择时钟，清除，时钟使能，输出使能等控制信号，外围控制总线提供了多达 12 个外围控制信号，在图 2.6 中分配。8 个输出使能信号，6 个时钟使能信号，2 个时钟信号，2 个清除信号。

2.1.3 互连资源

一般包括可编程的开关块，连接块，线等。开关块用来实现线与线之间互连，连接块用于和逻辑块端口连接。

线一般分为以下几种：

单长度线：用于 2 个相邻的逻辑块互连。

多长度线：用于实现相隔的逻辑块互连。但是单长线和多长度线因为要经过开关矩阵，每经过一次，延时增加 1 次，所以信号延时不可确定。

长线：跨越整个的行或者列。不经过开关阵列所以信号延时小，一般用于关键信号的传输。

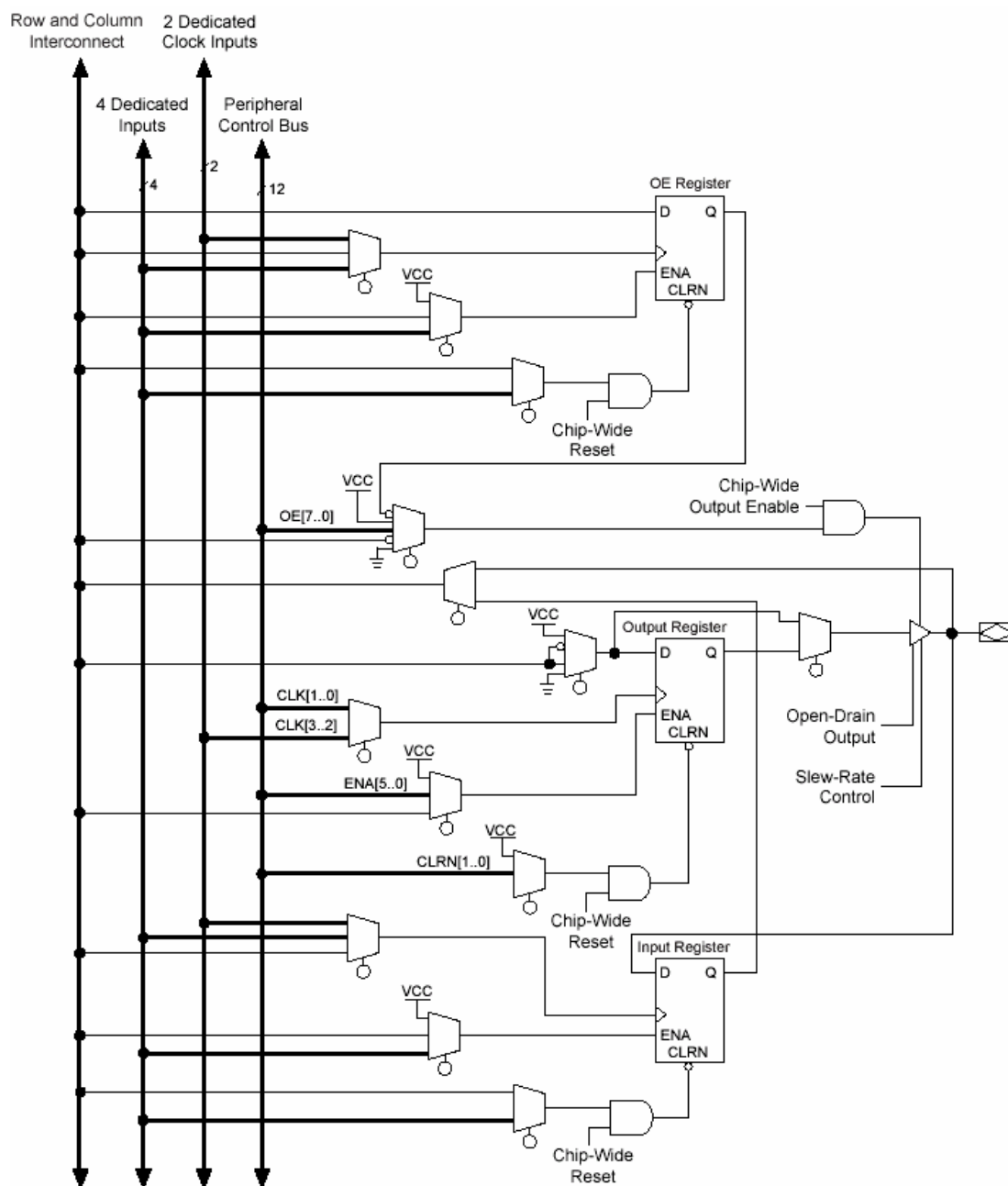


图 2.6 FLEX 10k I/O 模块

2.2 FPGA 结构相关术语定义

IOPAD: 一个 IO 输入端或输出端。

IOPAD Num: 每行或每列对应的 IOPAD 数目。

BLE: 基本的逻辑单元，模型可由一个 LUT 和一个 D 触发器组成，包含了组合逻辑和时序逻辑如图 2.4。

CLB: 复杂的可配置逻辑单元^[7]，由一个或一个以上的 BLE 组成，FPGA 在此基础上进行布局布线，如下图：

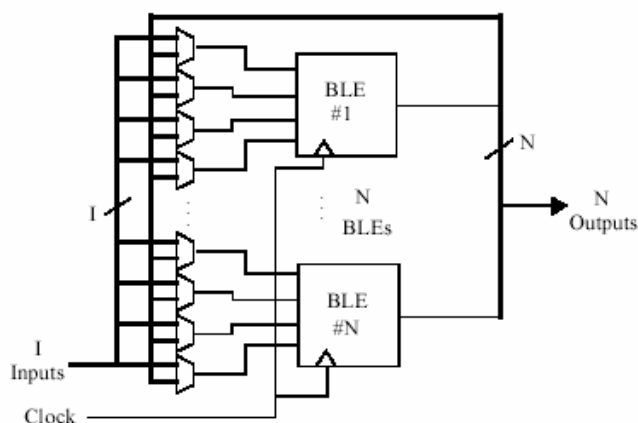


图 2.7 CLB 的结构

CLB Pin: CLB 的输入或输出端口, 有 inpin 和 outpin。

Chip Width: 芯片每一行的 CLB 的数目。

Chip Height: 芯片每一列的 CLB 的数目。

Wire Segment: 互连线, 是一种布线资源, 用于连接 CLB 的输入端口, 输出端口, IOPAD, 以及其他线。

Segment Length: 线长, 指的是一条线跨越 CLB 的个数, 有单长线(长度为 1, 跨过一个 CLB), 双长线(长度为 2, 跨过两个 CLB), 以及长线等。X 方向的长线长度是整个芯片的宽度, Y 方向的长线是整个芯片的高度。

Track: 由若干条 Segment 组成的, 如果该 Segment 长度是 L, 那么水平 Track 由 $\text{ceil}(\text{Chip Width}/L)$ 条 Segment 组成, 竖直 Track 由 $\text{ceil}(\text{Chip Height}/L)$ 条 Segment 组成。

Channel: CLB 之间或 IOPAD 和 CLB 之间的布线通道, 有 $\text{Chip Width}+1$ 条 Y 方向 Channel, 采取从下到上, 从左到右编码, 序号分别为 0, 1, 2... Chip Width ; 有 $\text{Chip Height}+1$ 条 X 方向 Channel, 采取从下到上, 从左到右编码, 序号分别为 0, 1, 2... Chip Height , 每条 Channel 有若干 Track。

Channel Width: Channel 布线通道的宽度, 如果为 W, 那么有 W 条 Track, 采取从下到上, 从左到右编码, 序号分别为 0, 1, 2... $\text{Channel Width}-1$ 。一般有 X 方向通道宽度, Y 方向通道宽度和 IO 通道宽度 3 种。

Connecting Box: 连接盒, 用于连接 Segment 和 CLB Pin, 如图 2.8^[7]。

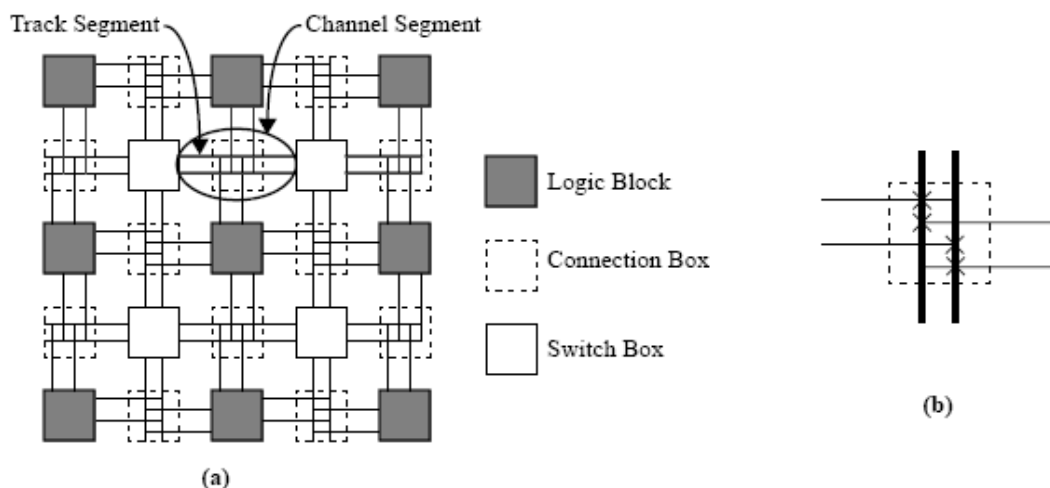


图 2.8 (a)CLB 结构 (b)连接盒

FC: 连接度, 是连接盒中用于描述连接情况, 有 IOPAD 的连接度, CLB input 的连接度, CLB output 的连接度, 如图 2.9 中, CLB 输入端口连接度 $fc_clbinputy=4$, CLB 输出端口的连接度 $fc_clboutputy=2$ 。

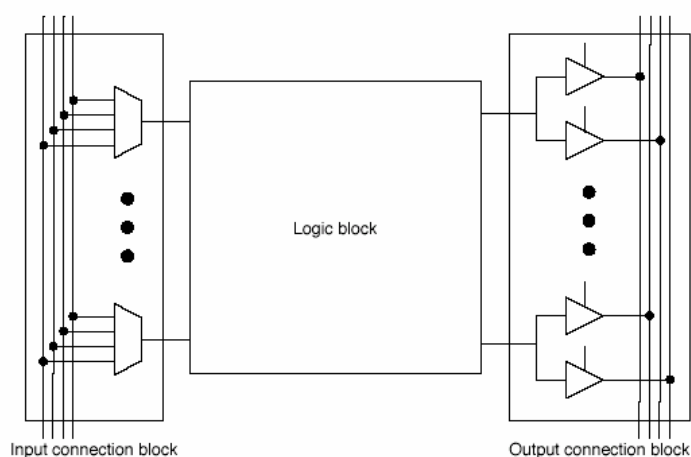


图 2.9 CLB 输入输出 Y 方向通道连接度

Switch Box: 开关盒, 用于连接 Segment 和 Segment, 如下图的开关盒含有 3 个连接点, 每个连接点有 6 个可编程开关(Programmable switch)。

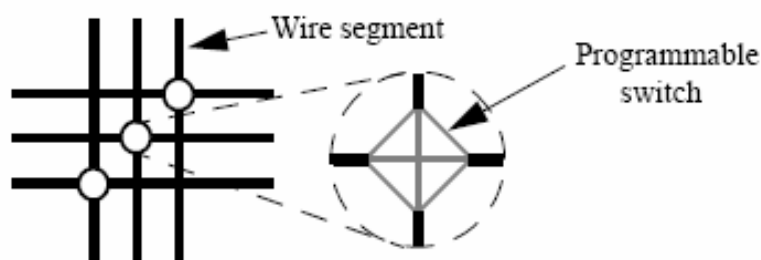


图2.10 开关盒

2.3 本章小结

本章介绍了现场可编程门阵列FPGA的结构，包括逻辑模块，互连模块，逻辑资源等，并对FPGA结构相关术语作了定义，方便了下一章介绍我们定义的FPGA结构描述方法。

第三章 结构描述式的 FPGA 建模方法

在布局布线阶段，需要知道FPGA的具体结构，为了能够不用修改布局布线工具而支持对不同的FPGA进行布局布线，需要对FPGA进行模型抽象，从而使布局布线工具和硬件FPGA结构分离。

FPGA结构的描述原来采用参数化^[6]的描述方法，参数化的描述方法是一个简单的抽象，没有完全表现出FPGA的细节，对FPGA各部分的抽象不明确，依靠布局布线工具的智能推理来确定FPGA的具体结构，且对很多FPGA结构特征不支持。

在参数化描述方法的基础上，我们参考虚拟现实语言VRML(Virtual Reality Modeling Language)中描述对象的语法，定义了一种FPGA结构描述方法。这种结构描述方法类似于VRML中结构体的表示方法，该描述方法能够支持更多的FPGA结构特征。该结构描述方法现共有10部分信息：芯片总体信息，逻辑单元信息，I/O端口信息，布图信息，开关信息，互连线信息，通道信息，开关盒信息，面积模型，延时模型。

定义的结构描述方法比参数化描述方法^[6]主要增加了：

- 1) 芯片名称；
- 2) 芯片大小描述，包含的逻辑单元数量；
- 3) X,Y 方向通道宽度及 IO 通道宽度描述；
- 4) CLB 逻辑单元的输入输出端口可连接精确布线通道；
- 5) 支持不同的 CLB 逻辑单元；
- 6) CLB 大小描述；
- 7) 对每一条 X,Y,IO 方向通道单独描述；
- 8) X,Y,IO 方向 Track 的开始点描述；
- 9) X,Y,IO 方向 Track 拥有的开关盒描述；
- 10) X,Y,IO 方向 Track 拥有的连接盒描述；
- 11) 增加芯片的布图描述，包含 CLB 布图和 IOPAD 布图；
- 12) 更多的开关盒样式支持，采用开关矩阵来描述复杂的开关盒。

结构描述方法采用#作为行注释标记，含有#的行，#后面是注释。为了将来和所开发的布局布线工具保持兼容，结构描述方法第一行必须是# V1.0用来说明当前结构描述方法的版本。对通道等的标记编码一律采用从左到右，从下到上的规则。结构描述方法语法格式基本是前面是一个关键字，后面接着带一个值的形式。num 后缀说明是个整型数，float 后缀说明是个浮点数，大写说明这个值只能选预定的几个值之一。[]说明是一个组合结构，有多个这样的[]并列。

3.1 芯片总体信息

芯片总体信息包含芯片名称，芯片宽度等信息，描述方法如下：

```
chipinfo{
    chip_name  "string"      #芯片名
    nx         width_num    #芯片宽度
    ny         height_num   #芯片高度
    iorat      rat_num      #每行或每列对应的IOPAD数
    wtrackclbx wtrackclbx_num #水平track宽度
    wtrackclby wtrackclby_num #竖直track宽度
    wtrackio   wtrackio_num  #IO通道宽度
    clbtypes   type_num      #逻辑单元的种类
    fc_io      num          #IO连接灵活度
    fc_clbinputx num        #CLB输入端口连接X方向上track灵活度
    fc_clbinputy num        #CLB输入端口连接Y方向上track灵活度
    fc_clboutputx num       #CLB输出端口连接X方向上track灵活度
    fc_clboutputy num       #CLB输出端口连接Y方向上track灵活度
}
```

`nx` 等单词都是关键字，`width_num` 是需要写的整型值。其他信息的格式也是这样设置，先是关键字，然后是值，后缀为 `num` 说明是个整数值。

图 3.1 中，`nx=6`，`ny=5`，`iorat=2`，`wtrackclbx=4`，`wtrackclby=4`，`wtrackio=4`，`clbtypes=2`。连接度如在图 2.9 中，`fc_clbinputy=4`，`fc_clboutputy=2`，其他的连接度类似。

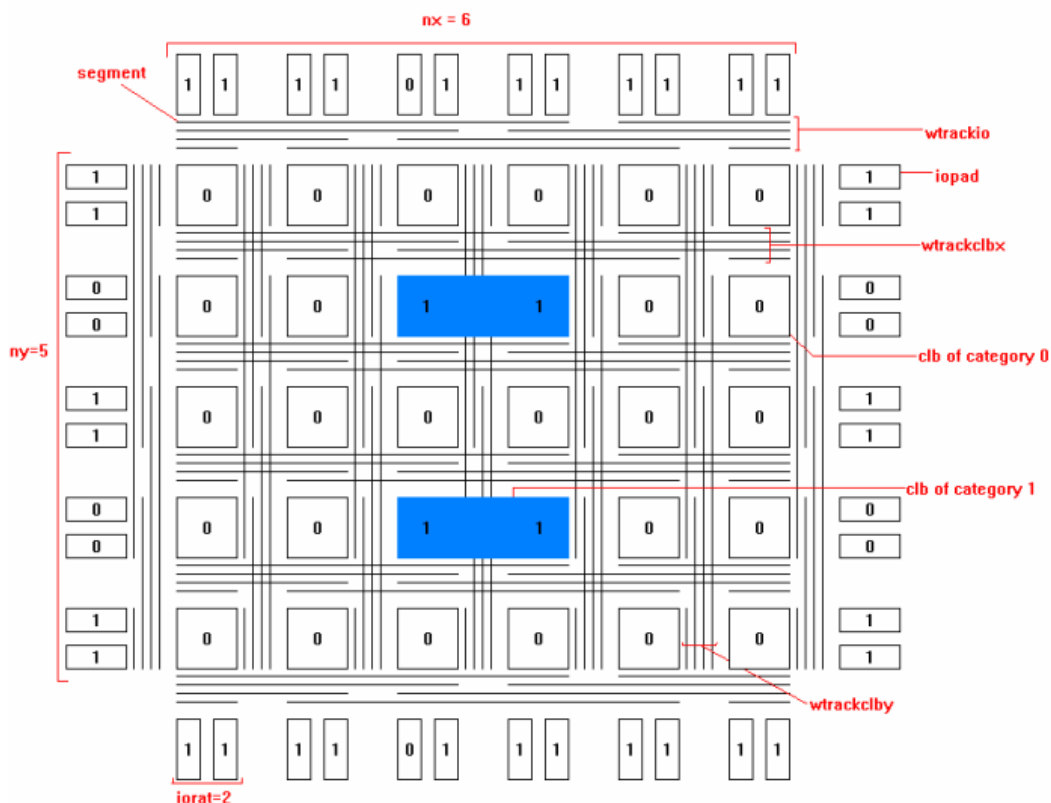


图 3.1 FPGA 总体结构

3.2 逻辑单元信息

在图3.2中是一个逻辑单元的外观图，端口0-7是输入端，用绿色表示；端口8，9是输出，用红色表示；端口10是时钟端，用黄色表示。每个端口都有4个位置，上，下，左，右。这里描述逻辑单元端口的位置信息是为了布局布线的时候选择相应位置的连线。

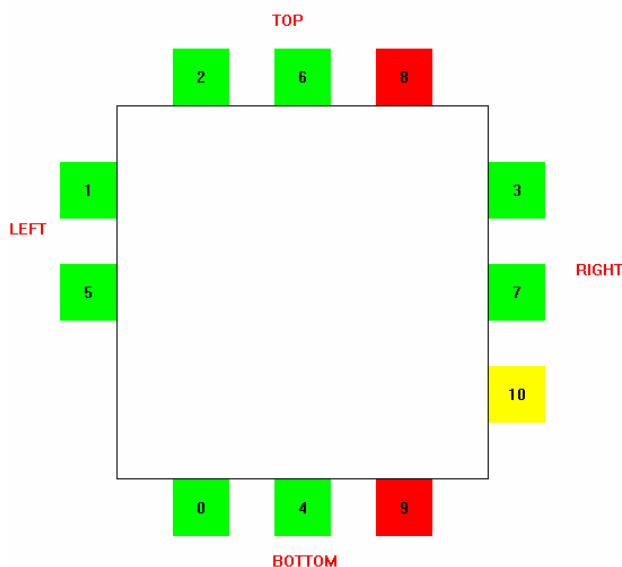


图 3.2 CLB 外观图

逻辑单元信息需要参考芯片总体信息中的 `clbtypes`，如果 `clbtypes>1`，那么有

不同的逻辑单元，需要定义多个逻辑单元信息。

每个逻辑单元信息语法格式如下：

```
clbinfo{
    name "block name"           #逻辑单元名字
    type  type_num              #逻辑单元的类型，从 0 开始
    sizex size_num              #逻辑单元的宽度
    sizey size_num              #逻辑单元的高度
    allpins{                    #定义逻辑单元的端口信息
        [type  PIN_TYPE         #类型为 INPIN 或 OUTPIN
            class type_num       #端口的逻辑等价类别
            position POS_TYPE    #端口所在逻辑单元的方向
            sub_part part_num     #端口所在的子部分
            connect_io_track num1 num2...#这个端口可以连接到的 IO track 号
            connect_clb_track num1 num2...#这个端口可以连接到的 clb track 号
            routetype ROUTE_TYPE #默认值 LOCAL
        ]
        [type  PIN_TYPE         #类型为 INPIN 或 OUTPIN
            class class_type     #端口的逻辑等价类别
            position POS_TYPE    #端口所在逻辑单元的方向
            sub_part part_num     #端口所在的子部分
            connect_io_track num1 num2... #这个端口可以连接到的 IO track 号
            connect_clb_track num1 num2... #这个端口可以连接到的 clb track 号
            routetype ROUTE_TYPE #默认值 LOCAL
        ]
        ...#继续定义其他端口
    }
}
```

clbinfo 中的 type 是逻辑单元的类型，类型取值范围从 0 到 clbtypes-1，type 用来标识不同类型的逻辑单元，比如一般逻辑单元，RAM 等。

sizex 和 sizey 是逻辑单元的宽度和高度，如果 $\text{sizex} \times \text{sizey} > 1$ ，那么这个逻辑单元的子部分编号从 0 到 $\text{sizex} \times \text{sizey} - 1$ 。

allpins 中的 type 是逻辑单元的 CLB IO 端口的类型，有 INPIN 和 OUTPIN2 种，标志它是输入和输出。PIN_TYPE 大写说明是只能写 INPIN 和 OUTPIN 不能写成整型值，其他信息也遵守这个规定。

class 是逻辑等价类别^[6]，如果 2 个 Pin 有相同的逻辑功能，且它们可以互换，

那么它们拥有相同的等价类别，用于布局布线的时候查找可替换的端口，Pin类别从 0 开始。

position 指定端口所在的方向，因为逻辑单元是矩形的，所以有 4 个方向，用 LEFT, RIGHT, TOP, BOTTOM 分别标识左，右，上，下。

sub_part 是 Pin 所在的子部分编号。因为线长是按单位逻辑单元表示，故当逻辑单元大小大于 1 时，需要用子部分序号进一步确定具体位置。

connect_io_track 是可以连接到的 IO track 序号，序号个数由 clbinfo 中的 fc_io 确定。

connect_clb_track 是可以连接到的 clb track 序号，有下面 4 种情况：

- 1) 如果 position 是 LEFT 或 RIGHT, type 是 INPIN, 那么序号个数由 clbinfo 中 fc_clbinputy 确定；
- 2) 如果 position 是 LEFT 或 RIGHT, type 是 OUTPIN, 那么序号个数由 clbinfo 中 fc_clboutputy 确定；
- 3) 如果 position 是 TOP 或 BOTTOM, type 是 INTPIN, 那么序号个数由 clbinfo 中 fc_clbinputx 确定；
- 4) 如果 position 是 TOP 或 BOTTOM, type 是 OUTPIN, 那么序号个数由 clbinfo 中 fc_clboutputx 确定。

routingtype 可以是 LOCAL 或 GLOBAL, LOCAL 类型参与实际布局布线，GLOBAL 类型不参与到布局布线，比如说全局时钟端口，不想让它参与布局布线时定义为 GLOBAL。

注意，可能有的时候 FPGA 的一个 Pin 对应 2 个位置,那么我们应该像下面这样描述这个 Pin，这是个 outpin，它在逻辑单元的位置为 BOTTOM 和 LEFT：

```
[ type OUTPIN
  class 4
  position BOTTOM
  sub_part 0
  position LEFT
  sub_part 0
  connect_io_track 0 1 3
  connect_clb_track 1 3 5 7 9
  connect_clb_track 0 2 4 6 8
  routingtype GLOBAL #可能是时钟或其它我们不想让它们参加布线的 Pin
]
```

3.3 I/O 端口信息

每行或每列对应chipinfo中的iorat个IOPAD,故需要对iorat个IOPAD描述它们可以连接到的IO track序号。可以连接的track数目由chipinfo中的fc_io决定。语法定义如下:

```
io_padinfo{
  connect_io_track{
    [num1 num2 num3 ...] #IOPAD 0 可以连接的 io track
    [num1 num2 num3 ...] #IOPAD 1 可以连接的 io track
    [num1 num2 num3 ...] #IOPAD 2 可以连接的 io track
    [num1 num2 num3 ...] #IOPAD 3 可以连接的 io track
    ...#其他的 IOPAD
  }
}
```

采用从左到右,从下到上编码 IOPAD 序号和 io track 序号,每个 IOPAD 都可以连接 fc_io 个 track。

3.4 布图信息

布图信息有 2 部分:逻辑单元(CLB)布图信息和 IOPAD 布图信息。

```
layout{
  clb_layout{
    num11 num12 num13 num14 ... #第一行 CLB 的类型, -1 表示不可用
    num21 num22 num23 num24 ... #第二行 CLB 的类型
    num31 num32 num33 num34 ... #第三行 CLB 的类型
    ...
  }
  io_layout{
    top_layout{ #上面的 IOPAD 布图信息
      [num1 num2 num3 num4 ...] #1 表示可以使用, 0 表示不可用
      [num1 num2 num3 num4 ...]
      ...
    }
    bottom_layout{ #下面的 IOPAD 布图信息
      [num1 num2 num3 num4 ...]
      [num1 num2 num3 num4 ...]
    }
  }
}
```



```

...
}
left_layout{    #左面的 IOPAD 布图信息
[num1 num2 num3 num4 ...]
[num1 num2 num3 num4 ...]
...
}
right_layout{   #右面的 IOPAD 布图信息
[num1 num2 num3 num4 ...]
[num1 num2 num3 num4 ...]
...
}
} #end of io_layout
} #end of layout

```

在图 3.1 中标明了一个 FPGA 的布图的数字，这个 FPGA 的 clb_layout 和 top_layout 如下：

```

clb_layout{
0 0 0 0 0 0
0 0 1 1 0 0
0 0 0 0 0 0
0 0 1 1 0 0
0 0 0 0 0 0
}
top_layout{
[1 1]
[1 1]
[0 1]
[1 1]
[1 1]
[1 1]
}

```

同样采用从左到右，从下到上的顺序编码。clb_layout[1][2]=1，clb_layout[1][3]=1，因为类型为 1 的 CLB 大小为 2*1，所以这 2 个 1 表示一个类型为 1 的 CLB，它有 2 个子部分，从左到右，从下到上编号为 0, 1，即 clb_layout[1][2] 对应子部分 0，clb_layout[1][3] 对应子部分 1。子部分号在逻辑单元信息的 Pin 的子部分 sub_part 需要用到。

IOPAD 布图信息有 4 部分，上面 IOPAD 布图信息，下面 IOPAD 布图信息，左边 IOPAD 布图信息，右边 IOPAD 布图信息。0 表示这个 IOPAD 不可用于布局布线，比如它是一个接地的端口或 FPGA 系统使用的端口等。

3.5 开关信息

开关用于连接互连线(wire segment)和互连线，或者连接 CLB 的输出端口到互连线。

```
Switchinfo{
  switch{
    type          type_num      #开关类型，从 0 开始
    isbuffered    BUFFER_TYPE   # FALSE 或者 TRUE
    rmetal        resistance_float #金属电阻，国际单位
    cin           capacitance_float #输入电容，国际单位
    cout          capacitance_float #输出电容，国际单位
    tdel          time_float     #内部延时，国际单位
  }
  ...#继续定义其他开关类型
}
```

图 3.3 中说明了 3 种开关类型，没有缓冲的传输开关，单向缓冲开关，双向缓冲开关。

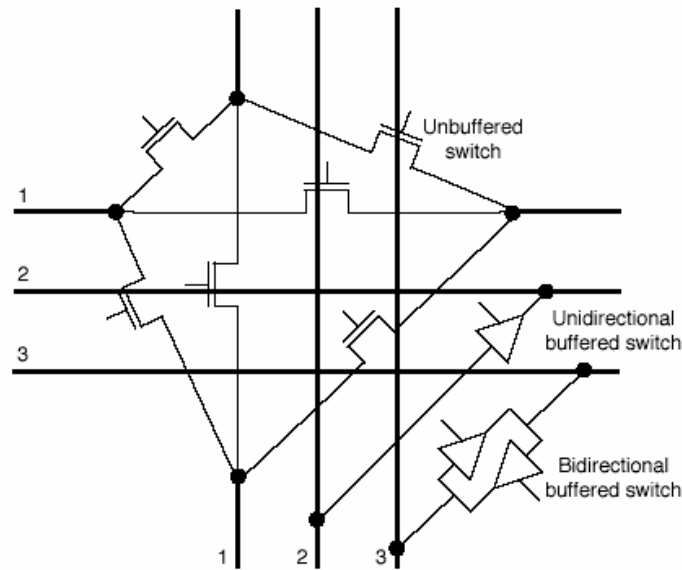


图 3.3 一种连接盒样式和三种开关说明

3.6 互连线信息

```
segmentinfo{
  segment{
    type          type_num      #线的类型，从 0 开始
    length        length_num    #线长度
    wire_switch    switch_type  #使用线连接的开关序号
    opin_switch    switch_type
```

```

#使用的输出端口到线的开关序号
rmetal      resistance_float
#线单位长度金属电阻，国际单位
cmetal      capacity_float
#线单位长度金属电容，国际单位
}
...#继续定义其它线信息
}

```

type 是设置线的类型，用来区分不同的线的类型。

length 是线的长度，如图 3.1 中的线的线长都是 4。

wire_switch 和 opin_switch 是使用的开关的序号，这些序号从 3.5 中定义的开关类型中选择。wire_switch 是这个类型的线与其他线连接所需要的开关类型，opin_switch 是输出端口到这个类型的线需要使用的开关的类型。

rmetal 是线单位长度每米的电阻，使用国际单位。

cmetal 是线单位长度的电容，使用国际单位。

3.7 通道信息

通道信息包括 3 部分，X 方向 CLB 通道信息，Y 方向 CLB 通道信息，I/O 通道信息。

因为连接盒用来连接线与逻辑单元的 IO Pin，所以长度为 L 的线在 L 个位置可以有连接盒(Connection Box)，开关盒用于线与线之间的连接，长度为 L 的线在 L+1 个位置可以有开关盒(Switch Box)。如在图 3.4 中，上面 2 个(track)通道采用的是长度为 2 的线，下面 2 个(track)通道采用的是长度为 4 的线。

图 3.4 中，长度为 2 的线可以连接 2 个连接盒，为全连接 2/2。它的开关盒有 2 个，只是在两头有连接，为 2/3 连接。

图 3.4 中，长度为 4 的线有 2 个连接盒，为 2/4 连接。它的开关盒有 5 个，为 5/5 全连接。

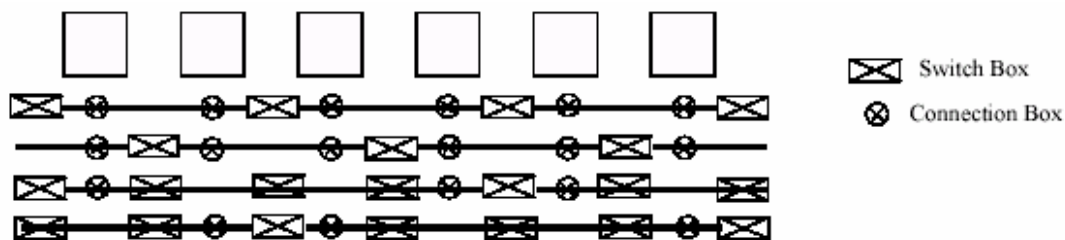


图 3.4 线的开关盒和连接盒情况

再如图 3.5 中长度为 5 的线可能有的几种连接盒情况：

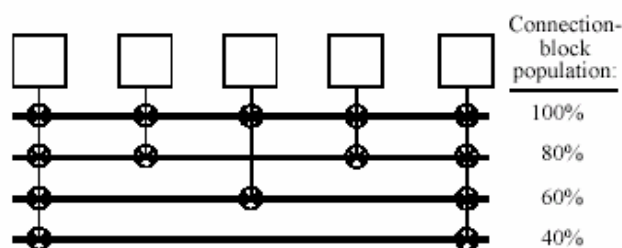


图 3.5 长度为 5 的线的可能的连接盒

通道的开始点是所使用的第一条线的线长。如图 3.6 中使用的是长度为 4 的线。开始点从下到上依次为 1, 2, 3, 4。在实际的 FPGA 中为了有更好的布通性, 开始点一般都不相同。

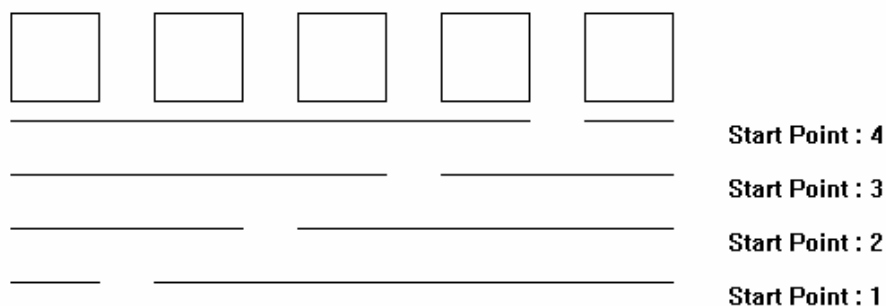


图 3.6 通道的开始点

通道信息的描述方法如下:

```
trackinfo {
    x_trackinfo { #X 方向 CLB 通道信息
        [ track track_num #通道编号, 从 0 开始
            start_point num #通道开始点
            cb "string" #连接盒分布如: "1111"
            sb "string" #开关盒分布如: "10101"
            segment_type segment_num #引用的线的类型
        ]
        #其他 X 方向通道
    }
    y_trackinfo { #Y 方向 CLB 通道信息
        [ track track_num #通道编号, 从 0 开始
            start_point num #通道开始点
            cb "string" #连接盒分布如: "1111"
            sb "string" #开关盒分布如: "10101"
            segment_type segment_num #引用的线的类型
        ]
    }
}
```

```

#其他 y 方向通道
}
io_trackinfo{      #IO 通道信息
[   track   track_num   #通道编号, 从 0 开始
    start_point   num #通道开始点
    cb   "string"   #连接盒分布如: "1111"
    sb   "string"   #开关盒分布如: "10101"
    segment_type   segment_num #引用的线的类型
]
#其他 IO 通道
}
}

```

segment_type 是这个通道使用的线的类型, 这个类型对应于 3.6 中描述的线的类型。cb 是连接盒的分布, 如果该类型线长度为 4, 那么连接盒分布“1111”表示全连接, “1001”表示 2/4 连接, 1 表示该位置有连接盒, 0 表示该位置没有连接盒。sb 是开关盒的分布, 如果该类型线长度为 4, 那么开关盒分布“11111”表示全连接, “10101”表示 3/5 连接。同样, 1 表示该位置有连接盒, 0 表示该位置没有连接盒。

3.8 开关盒信息

共有 5 种开关盒样式来描述 CLB 间的开关盒样式和 PAD 与 CLB 间的开关盒样式:

- 1) CLB 间的开关盒样式 clb_switchbox
- 2) 左边 PAD 与 CLB 样式 left_switchbox
- 3) 右边 PAD 与 CLB 样式 right_switchbox
- 4) 上边 PAD 与 CLB 样式 top_switchbox
- 5) 下边 PAD 与 CLB 样式 bottom_switchbox

在每一种开关盒样式中又有 12 种子样式:

- 1) 左边通道到右边通道开关盒 left_to_right_switchbox
- 2) 左边通道到上边通道开关盒 left_to_top_switchbox
- 3) 左边通道到下边通道开关盒 left_to_bottom_switchbox
- 4) 右边通道到左边通道开关盒 right_to_left_switchbox
- 5) 右边通道到上边通道开关盒 right_to_top_switchbox
- 6) 右边通道到下边通道开关盒 right_to_bottom_switchbox
- 7) 上边通道到左边通道开关盒 top_to_left_switchbox

- 8) 上边通道到右边通道开关盒 top_to_right_switchbox
- 9) 上边通道到下边通道开关盒 top_to_bottom_switchbox
- 10) 下边通道到左边通道开关盒 bottom_to_left_switchbox
- 11) 下边通道到右边通道开关盒 bottom_to_right_switchbox
- 12) 下边通道到上边通道开关盒 bottom_to_top_switchbox

开关盒描述信息语法定义如下:

```
Switchboxinfo{
    switchbox_type BOX_TYPE
        #可以为 CUSTOM, SUBSET,WILTON 或 UNIVERSAL
    clb_switchbox{ #CLB 之间的开关盒
        left_to_right_switchbox{
            [num1 num2 ...] #左边通道 track 0 能够连接到的右边通道号
            [num1 num2 ...] #左边通道 track 1 能够连接到的右边通道号
            ...
        }
        left_to_top_switchbox{
            [num1 num2 ...] #左边通道 track 0 能够连接到的上边通道号
            [num1 num2 ...] #左边通道 track 1 能够连接到的上边通道号
            ...
        }
        #其他的 10 种子样式
    }
    left_switchbox{ #左边 PAD 与 CLB 间的开关盒
        #12 种子样式
    }
    right_switchbox{ #右边 PAD 与 CLB 间的开关盒
        #12 种子样式
    }
    #其他 2 种样式
}
```

switchbox_type 是开关盒的类型, 这里设定了 4 种类型可以选择,CUSTOM, SUBSET,WILTON 和 UNIVERSAL。如果设置为 CUSTOM,那么是自定义类型, 需要写 clb_switchbox, left_switchbo, right_switchbox, top_switchbox, bottom_switchbox 这 5 种开关盒样式。其他 3 种类型 SUBSET,WILTON 和 UNIVERSAL 是这里默认支持的 3 种已经定义的格式。选择这 3 种类型, 那么就不用定义 5 种开关盒样式

了, 如果不设置开关盒信息, 假定布局布线算法默认采用已经定义的 SUBSET 格式。但是这 3 种已经定义的格式只支持 X 方向的通道数和 Y 方向的通道数相等, 且连接度为 3 的情况, 连接度为 3 是每条通道只能连接另外 3 条通道。若要实现更复杂的开关盒样式, 需要选择 CUSTOM, 然后自己写 5 个开关盒样式。

在图 2.10 中的开关盒样式就是 SUBSET 格式, 这个图中, 水平的通道和竖直的通道都是 3, 0 号左边通道可以连接 0 号右边通道, 0 号上边通道, 0 号下边通道等 3 个通道, 其他类似, 所以连接度为 3, 且要求 X,Y 方向的通道数相等。对应的左边到右边的通道描述为:

```
left_to_right_switchbox{
[0] #左边 0 号通道连接右边 0 号通道
[1] #左边 1 号通道连接右边 1 号通道
[2] #左边 2 号通道连接右边 2 号通道
}
```

3.9 面积模型

面积模型主要用来估算布局布线算法中所使用的资源所占用的面积。这里采用等效成使用了多少个晶体管来表示。

```
areamodel{
    r_minw_nmos    resistance_float    #最小宽度 nmos 电阻
    r_minw_pmos    resistance_float    #最小宽度 pmos 电阻
}
```

3.10 延时模型

延时模型主要用来分析电路的时序, 和指导 FPGA 布局布线算法进行布局布线时考虑延时的因素, 来提高布局布线算法的质量。这里主要是对 IOPAD 和 CLB 建立模型, 线, 开关等的延迟已经在前面定义线和开关的时候定义了。

```
delaymodel{
    t_ipad         time_float          #通过输入 I/O 模块的延时
    t_opad         time_float          #通过输出 I/O 模块的延时
    clb_delaymodel {
        type                type_num                #CLB 的类型
        subblock_nums       block_num                #子块的数目
        lut_size            num                    #查找表的大小
        c_ipin_cblock       capacity_float
    }
```

```

#布线通道和连接盒间缓冲器输入电容
    t_ipin_cblock    time_float
#通道通过缓冲器和连接块(如多路选择器)到逻辑块输入端口延时
    t_sblk_opin_to_sblk_ipin    time_float
#子块输出到子块输入延迟
    t_clb_ipin_to_sblk_ipin    time_float
#CLB 输入到子块输入延迟
    t_sblk_opin_to_clb_opin    time_float
#子块输出端口到 CLB 输出延迟
    subblock_delaymodel{
    [
        t_comb    time_float    #组合延迟
        t_seq_in    time_float    #触发器时钟建立延迟+组合延迟
        t_seq_out    time_float    #触发器到子块输出端口延迟
    ]
    subblock_delaymodel{
    [
        t_comb    time_float    #组合延迟
        t_seq_in    time_float    #触发器时钟建立延迟+组合延迟
        t_seq_out    time_float    #触发器到子块输出端口延迟
    ]
    ...    #其他子块
    }
    #如果有多种 CLB，继续定义其它 CLB 延时模型
}

```

c_ipin_cblock 是布线通道和连接盒间缓冲器输入电容，如图 3.7

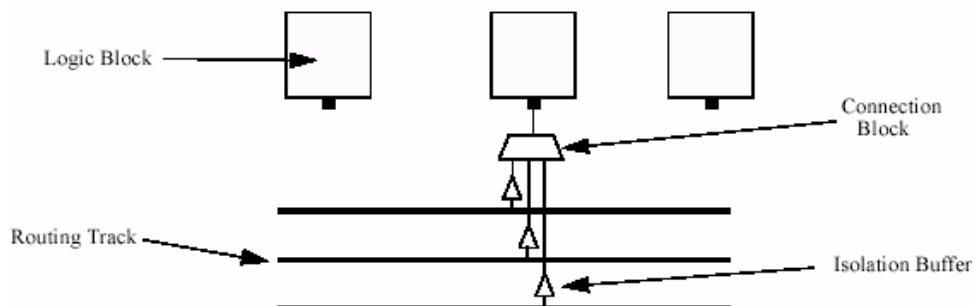


图 3.7 布线通道道逻辑单元的连接结构

图 3.8 是子块输出到子块输入延迟，CLB 输入到子块输入延迟，子块输出端

口到 CLB 输出延迟。

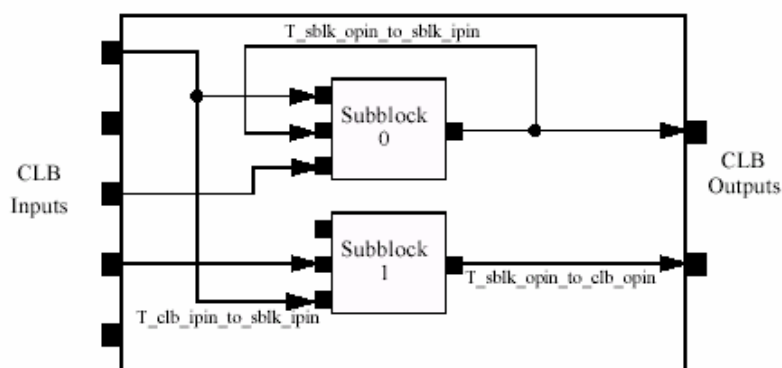


图 3.8 CLB 内输入输出端口与子块端口的延时

子块内部延时使用 t_comb , t_seq_in , t_seq_out 。当使用组合逻辑时, 使用 t_comb , 当使用时序逻辑时, 使用 t_seq_in 和 t_seq_out 。

t_comb : 从子块输入到子块输出延时, 用于组合逻辑。当子块仅用到组合逻辑时, 就使用组合逻辑延时来估算这个子块的内部延时。

t_seq_in : 时序逻辑下, 子块输入端到触发器存储单元延时。子块用于时序模式下当网表连接时钟端口到一些信号。假如子块是个简单的触发器。那么 t_seq_in 就是建立时间。假如是 LUT 和触发器, 那么 T_seq_in 就是 LUT 延时加上建立时间。

t_seq_out : 时序逻辑下, 从子块存储单元到子块输出端口的延时。子块用于时序模式下当网表连接时钟端口到一些信号。假如子块有个触发器连接到输出端口, 那么 t_seq_out 是触发器的存储单元到输出延时。

3.11 本章小结

这一章定义了 FPGA 结构描述方法, 该结构描述方法能够对 FPGA 结构进行建模, 描述了 FPGA 的 10 个关键特征。结构描述方法将成为一个中间层, 为布局布线工具提供服务, 当对不同 FPGA 芯片进行布局布线时, 仅需要编写相应的结构描述文件。下一章将讨论在此结构描述文件基础上的 FPGA 布局布线算法和划分算法。

第四章 划分算法和布局布线算法的分析与研究

4.1 FPGA 支持软件设计流程

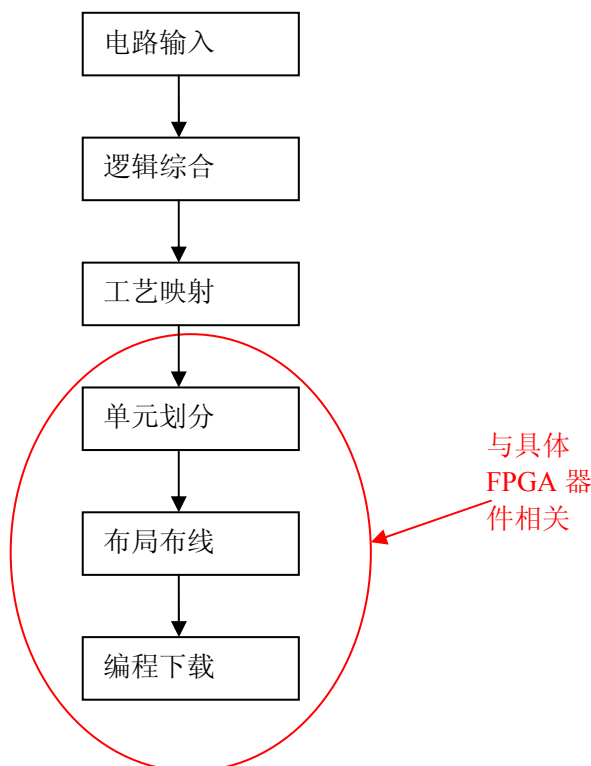


图 4.1 FPGA 支持软件设计流程的六个关键步骤

- 1) 电路输入是把用户输入的 VHDL, Verilog, 电路原理图等检查语法错误后转换成逻辑网表。
- 2) 逻辑综合是把逻辑网表进行逻辑优化, 考虑面积减少, 延迟减少, 逻辑化简, 功耗降低等, 化简后生成优化网表。
- 3) 工艺映射是把优化网表进行跟工艺相关的映射转换, 比如转化成含有 LUT(查找表)和 flip-flop (触发器)的网表, 最后生成映射后的网表。
- 4) 单元划分要考虑把映射后的网表划分到实际的 FPGA 芯片逻辑单元, 把映射后的网表中的 LUT, flip-flop 等合并组合成实际 FPGA 芯片逻辑单元, 生成逻辑单元网表。
- 5) 布局布线把生成的单元网表进行确定每个单元在芯片中的位置(布局), 确定每个单元之间如何连线(布线)。
- 6) 编程下载分 2 步, 编程是把布局布线的结果生成可以写入到芯片的位流文件, 下载是把位流文件写入 FPGA 芯片中。

4.2 FPGA 划分和布局布线问题

4.2.1 划分算法回顾

RASP^[8]是一个基于SRAM的FPGA综合系统,它可以把电路划分成基于查找表LUT的逻辑单元。它创建一个图,图中顶点表示LUT,如果2个顶点之间有边表示这2个LUT可以放到同一个逻辑单元中。如果2个顶点之间没有边,可能是因为违反了逻辑单元的某些约定比如是超过了逻辑单元的最大输入数目。VPACK^[1]输入LUT和触发器(latch)的网表,输出如图4.2的逻辑单元网表。它把图2.4的基本逻辑单元(BLE)组合成复杂的逻辑单元(CLB),CLB就是一种FPGA使用的逻辑单元。一个BLE包含一个LUT和一个Latch,而一个CLB包含多个的BLE。

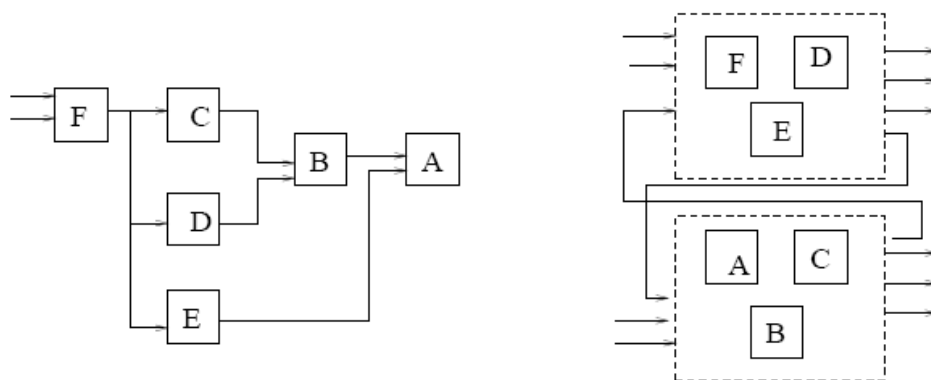


图 4.2 LUT 和 Latch 网表(左), 逻辑单元网表(右)

设定 CLB 含有 N 个 BLE, 有 I 个输入, N 个输出, M 个时钟, LUT 的大小为 K 。VPACK 尝试在满足下面的条件下把尽量多的 BLE 放到一个 CLB 中, 来减少使用的 CLB 的个数。

- 每个 CLB 中最多有 N 个 BLE
- 每个 CLB 中最多使用 I 个输入
- 每个 BLE 最多使用 K 个输入
- 每个 CLB 最多有 M 个时钟

VPACK 分 2 步: 首先把 LUT 和 Latch 组合成 BLE, 然后把 BLE 组合成 CLB。

4.2.2 FPGA 布局问题

FPGA 布局的任务是在芯片的可变模块中安排所有的逻辑单元的位置。理想情况下布局阶段的目标是:

- 1) 确保布线器能够完成布线
- 2) 最小化关键网络延时
- 3) 使芯片尽量密集

还可能会有一些附加目标:

- 4) 最小化功耗
- 5) 最小化信号间的串扰

这些任务很难用算法的解来定义, 满足要求比较困难, 所以目前的布局工具一般采用更确定且可达到的准则, 最常用的布局目标是下面的一个或多个:

- a) 使估计的互连总长度最小
- b) 符合关键网络的时序要求
- c) 使互连的拥塞最小

这些目标中的每一个或多或少都会影响到另一个, 所以一般要折中考虑。

布局问题是NP难问题, 商用CAD工具中存在两类布局算法, 早期的结构式布局算法和迭代式布局算法。结构式布局算法是用一组规则达到结构化布局。比较常用的方法是最小割算法, 最小割布局算法^[9]应用了逐次划分的方法。现在最常用的是迭代式布局算法, 首先产生一个初始解, 然后用迭代算法来改进它。

模拟退火算法(Simulated Annealing algorithm)是目前使用最广泛的迭代式布局算法, Kirkpatrick, Gerlatt和Vecchi最先应用模拟退火算法来解VLSI问题^[10]。从那以后的经验表明模拟退火算法通常需要很慢的冷却进度, 这样就需要很长的CPU运行时间^[11]。实验证明基于最小割的布局布线算法比模拟退火算法快, 但是模拟退火布局算法可以获得更好的布局质量。

VPR(Versatile Place and Route)^[12]是一个通用布局布线工具, 它在简单结构的基础上执行布局布线操作, 它布局采用的是模拟退火算法, 该布局算法是时序驱动布局算法^[13], 即在布局的过程中同时考虑关键路径延时和布局总线长。由于VPR是一个开源工具, 后来就有很多人进行VPR中算法的改进并且和原算法结果比较, 来验证他们算法的好坏。

目前国外关于VPR布局算法的改进主要集中于修改模拟退火布局算法来提高布局的速度, 因为FPGA支持软件运行一个大的电路需要几十分钟到几个小时, 布局布线占用了大概 40%的运行时间。初始布局位置方法^[14]采用一个好的初始布局位置的方法来加速模拟退火布局, 分治布局^[15]采用基于分治的时序驱动布局算法来加速布局, 并行布局^[16]和^[17]采用并行计算的方法来加速模拟退火布局算法。这些方法基本都是在降低解的质量的前提下, 把布局算法的运行速度加快几倍到十几倍。

4.2.3 FPGA 布线问题

FPGA 的布线的任务是安排逻辑单元之间如何通过布线通道和布线开关进行连线。布线目标是下面的一个或者多个:

- a) 使互连总长度最小

布线,从而综合考虑所有布线资源,获得比较好的效果。使用组合的布线算法如TRACER^[28]是一个考虑时序的布线工具,GBP(Greedy Bin Packing)^[29]是一个考虑布通性的布线工具。

PathFinder^[30]是一个既考虑时序又考虑布通性的算法,VPR^[12]的布线算法是在PathFinder的基础上改进的多次迭代算法。由于VPR是个开源的工具,它的布线算法能够提供较好的布线质量,但是需要较多的运行时间,有很多人在此基础上改进它以加快它的运行速度,如改进算法^[31]少用了13倍的内存,运行速度加速了10倍,仅需要比VPR多用10%的布线资源。

4.3 VPACK 和 VPR 的不足

VPACK^[1]是一个划分工具,可以把工艺映射后的网表划分成FPGA逻辑单元网表,但是在划分的过程中没有提取电路的真值表信息,目前FPGA一般采用基于SRAM工艺的LUT(查找表)结构,我们在VPACK算法的基础上,完成一个改进划分网表工具使在划分逻辑单元的同时,能够得到真值表信息,并且产生LUT的配置文件。

VPR是个比较出色的布局布线工具^[12],Altera公司收购了VPR的布局布线算法提高了他们布局布线软件的布通性,VPR的4.30版是对外开源的最终版。

VPR4.30支持的是单一的逻辑单元,很多结构细节不支持,对开关盒,连接盒,互连线,布图等没有做详细的描述,且它仅是在简单的结构上比较算法的优劣,不去考虑如何应用于实际的FPGA芯片,布局布线下一阶段编程所需要的必要的数据没有提供。

但是现在商业上的FPGA包含更加复杂的结构,所以我们在VPR4.30布局布线算法的基础上采用FPGA结构描述方法对FPGA芯片建立模型,增加了更多的结构特征的支持,能够反映真实的FPGA芯片,并且改进布局算法加快模拟退火算法的运行速度,改进布线算法使它能够同时考虑外部布线和内部布线,最终能够生成编程下载所必需的布局文件,布线文件和内部连接文件。

前一章描述了FPGA结构描述方法对FPGA建立模型,FPGA结构描述方法可以表示不同的逻辑单元,还增加了布图信息,更加灵活的开关盒样式,指定逻辑单元的端口可以连接到的通道等,可以建立一个FPGA结构描述文件来描述FPGA芯片的具体结构。

改进划分网表工具的输入是工艺映射后网表文件,输出是逻辑单元网表文件和LUT配置文件。

改进布局布线工具的输入是FPGA结构描述文件,逻辑单元网表文件,输出是布局文件,布线文件,内部连接文件,关键路径文件,布线通道使用统计文件等。

LUT配置文件, 布局文件, 布线文件, 内部连接文件这4个文件为布局布线的下一步编程下载提供必要的信息, 可以生成位流文件, 如图4.4:

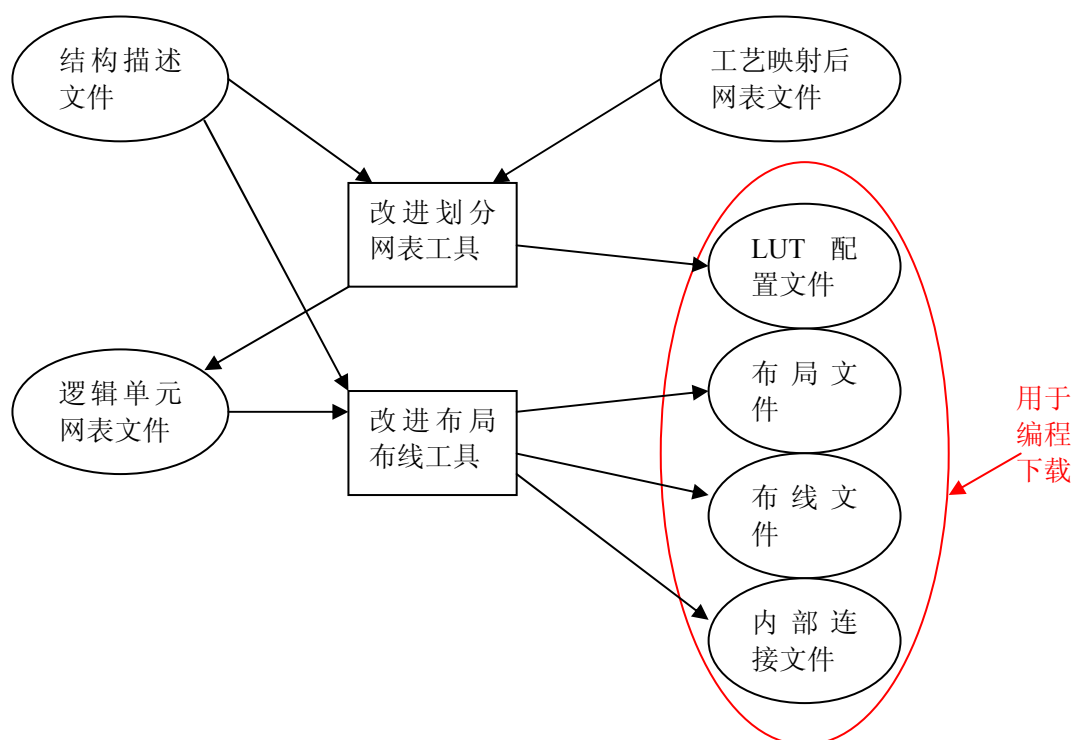


图 4.4 改进划分网表工具和布局布线工具的输入和输出

4.4 改进的划分网表工具

我们的改进划分网表工具是在 VPACK 的基础上进行改进, 因为 VPACK 是个划分工具, 仅考虑如何把 LUT 和 flip-flop 划分到一起, 没有考虑读入电路的真值表等信息, 没有生成每个 CLB 中 LUT 的配置。但是我们不仅仅做算法的研究, 还要给编程下载阶段提供需要的电路配置信息, 所以我们要从电路中读取真值表信息, 然后转换成编程下载可以使用的 LUT 配置信息。

4.4.1 改进的划分算法

改进的划分算法步骤如下:

(1)读取真值表。从工艺映射后的网表文件中读取真值表信息, 同时记录下每个 LUT 的名字和相关的真值表信息。

(2)生成基本逻辑单元 BLE。

a) 对于包含 LUT 和 flip-flop 的一般电路工艺映射后的网表, 把只有一个扇出(fanout)的 LUT 并且它的输出直接反馈到一个 flip-flop 的输入, 那么这个 LUT 和这个 flip-flop 就可以合并为一个 BLE。这个 BLE 的名字是这个 Latch 的名字, 真值表配置是 LUT 的真值表。

b) 如果 LUT 不能够和 flip-flop 合并,那么这个 BLE 的名字用 LUT 的名字表示。

c) 如果 flip-flop 作为一个 BLE,那么 BLE 的名字用 flip-flop 的名字表示。

(3)把 BLE 组合成 CLB。采用 VPACK 中的算法,算法首先采用贪心法向当前的 CLB 中加入 BLE,直到达到了 CLB 中的 BLE 要求的最大数目。然后重新创建一个 CLB,继续重复这个过程。但是在给 CLB 选择第一个 BLE 的时候要选择具有最多的输入端口的 BLE,然后再选择第 2 个以后的 BLE 的时候,选择具有和当前 CLB 有最大吸引度的 BLE。BLE 和 CLB 的吸引度(attraction)大小是 BLE 连接的网和 CLB 连接的网的相同的网数目。具体算法见图 4.5。

```

Let:UnclusteredBLEs be the set of BLEs not contained in any cluster
    C be the set of BLEs contained in the current cluster
    LogicClusters be the set of clusters (where each cluster is a set of BLEs)

UnclusteredBLEs = PatternMatchToBLEs (LUTs, Registers);
LogicClusters = NULL;

while (UnclusteredBLEs != NULL) { /* More BLEs to cluster */
    C = GetBLEwithMostUsedInputs (UnclusteredBLEs);
    while (|C| < N) { /* Cluster is not full */
        BestBLE = MaxAttractionLegalBLE (C, UnclusteredBLEs);
        if (BestBLE == NULL) /* No BLE can be added to cluster */
            break;
        UnclusteredBLEs = UnclusteredBLEs - BestBLE;
        C = C ∪ BestBLE;
    }
    LogicClusters = LogicClusters ∪ C;
}

```

图 4.5 BLE 组合成 CLB 算法

(4)真值表转换并生成相应的 CLB 配置文件。

4.4.2 一个简单的文件转换实验

为了说明问题,下面举一个 4 位全加器的例子,如下是输入的是一个工艺映射后网表文件,其功能是一个 4 位全加器,文件格式是 blif 格式。

```

.model full_add4
.inputs a0 a1 a2 a3 b0 b1 b2 b3 cin
.outputs sum0 sum1 sum2 sum3 cout
.names a0 b0 cin sum0
001 1
010 1
100 1
111 1

```

```

.names a0 b0 cin [1]
11- 1
1-1 1
-11 1
.names a1 b1 [1] sum1
001 1
010 1
100 1
111 1
.names a1 b1 [1] [2]
11- 1
1-1 1
-11 1
.names a2 b2 [2] sum2
001 1
010 1
100 1
111 1
.names a2 b2 [2] [3]
11- 1
1-1 1
-11 1
.names a3 b3 [3] sum3
001 1
010 1
100 1
111 1
.names a3 b3 [3] cout
11- 1
1-1 1
-11 1
.end

```

经过我们修改的划分工具的处理后，输出的逻辑单元网表文件如下，使用了3个可配置逻辑单元：clb [1]， clb sum1， clb sum1。

```

.input a0 pinlist: a0
.input a1 pinlist: a1
.input a2 pinlist: a2
.input a3 pinlist: a3
.input b0 pinlist: b0
.input b1 pinlist: b1
.input b2 pinlist: b2
.input b3 pinlist: b3
.input cin pinlist: cin
.output out:sum0 pinlist: sum0
.output out:sum1 pinlist: sum1
.output out:sum2 pinlist: sum2
.output out:sum3 pinlist: sum3
.output out:cout pinlist: cout
.clb [1]

```

```

pinlist: a0 b0 cin a1 b1 a2 b2 a3 b3 open [1] [2] [3] sum3 open
subblock: [1] 0 1 2 open 10 open
subblock: [2] 3 4 ble_0 open 11 open
subblock: [3] 5 6 ble_1 open 12 open
subblock: sum3 7 8 ble_2 open 13 open
.clb sum1
pinlist: a1 b1 [1] a2 b2 [2] a3 b3 [3] open sum1 sum2 cout open open
subblock: sum1 0 1 2 open 10 open
subblock: sum2 3 4 5 open 11 open
subblock: cout 6 7 8 open 12 open
.clb sum0
pinlist: a0 b0 cin open open open open open open open sum0 open open open open
subblock: sum0 0 1 2 open 10 open

```

如下是输出的 LUT 配置文件，表示了逻辑单元内部 SRAM 位的配置。

```

.clb [1]
LUT 0:    033f
LUT 1:    033f
LUT 2:    033f
LUT 3:    3cc3
.clb sum1
LUT 0:    3cc3
LUT 1:    3cc3
LUT 2:    033f
.clb sum0
LUT 0:    3cc3

```

逻辑单元网表是把电路划分成 CLB 的网表文件，该 CLB 有 10 个输入，4 个输出和 1 个时钟，这里 1 个 CLB 中包含了 4 个 BLE，没有使用的 BLE 不列出。

sum0 是本位和，cin 是低位的进位，a0, b0 是一位数的加数，[1]是产生的进位。在 CLB sum0 中 a0, b0, cin 使用了该 CLB 的前 3 个输入端口(编号 0, 1, 2)，sum0 使用了 CLB 的第一个输出端口(编号为 10)，未使用的端口都是用 open 标记。a0, b0, cin, sum0 使用了 CLB sum0 的第一个 BLE 即 LUT 0，其 LUT 配置为 3cc3。同样，a0, b0, cin, [1]使用了 CLB [1]的第一个 BLE 即 LUT 0，其 LUT 配置为 033f。

LUT 的配置采用 16 进制。因为是 4 输入的 LUT，所以 LUT 的配置需要 16 位来存储。由于是组合逻辑，BLE 只使用了 LUT，没有使用 flip-flop。根据 CLB sum0 的 LUT0 配置和 CLB [1]的 LUT0 的配置，可得到表 4.1 所示 a0, b0, cin, sum0, [1]的真值表。由于 LUT 具有 4 个输入，没有使用的输入端可以为任意值，是无关项，这里用 X 表示没有使用的 LUT 输入端。

表 4.1 真值表

a0	b0	cin	X	sum0	[1]
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	1	0
0	0	1	1	1	0
0	1	0	0	1	0
0	1	0	1	1	0
0	1	1	0	0	1
0	1	1	1	0	1
1	0	0	0	1	0
1	0	0	1	1	0
1	0	1	0	0	1
1	0	1	1	0	1
1	1	0	0	0	1
1	1	0	1	0	1
1	1	1	0	1	1
1	1	1	1	1	1

4.5 改进的布局算法

4.5.1 模拟退火算法

模拟退火算法来源于固体退火原理，将固体加温至充分高，再让其徐徐冷却，加温时，固体内部粒子随温升变为无序状，内能增大，而徐徐冷却时粒子渐趋有序，在每个温度都达到平衡态，最后在常温时达到基态，内能减为最小。根据 Metropolis 准则，粒子在温度 T 时趋于平衡的概率为 $\exp(-\Delta E/kT)$ ，其中 E 为温度 T 时的内能， ΔE 为其改变量， k 为 Boltzmann 常数。用固体退火模拟组合优化问题，将内能 E 模拟为目标函数值 f ，温度 T 演化成控制参数 t ，即得到解组合优化问题的模拟退火算法：由初始解 i 和控制参数初值 t 开始，对当前解重复“产生新解→计算目标函数差→接受或舍弃”的迭代，并逐步衰减 t 值，算法终止时的当前解即为所得近似最优解，这是基于蒙特卡罗迭代求解法的一种启发式随机搜索过程。退火过程由冷却进度表(Cooling Schedule)控制，包括控制参数的初值 t 及其衰减因子 Δt 、每个 t 值时的迭代次数 L 和停止条件 S 。

模拟退火算法可以分解为解空间、目标函数和初始解三部分。

模拟退火的基本思想:

- (1) 初始化: 初始温度 T (充分大), 初始解状态 S (是算法迭代的起点), 每个 T 值的迭代次数 L
 - (2) 对 $k=1, \dots, L$ 做第(3)至第 6 步:
 - (3) 产生新解 S'
 - (4) 计算增量 $\Delta t' = C(S') - C(S)$, 其中 $C(S)$ 为评价函数
 - (5) 若 $\Delta t' < 0$ 则接受 S' 作为新的当前解, 否则以概率 $\exp(-\Delta t'/T)$ 接受 S' 作为新的当前解.
 - (6) 如果满足终止条件则输出当前解作为最优解, 结束程序.
- 终止条件通常取为连续若干个新解都没有被接受时终止算法.
- (7) T 逐渐减少, 且 $T > 0$, 然后转第 2 步.

模拟退火算法新解的产生和接受可分为如下四个步骤:

第一步是由一个产生函数从当前解产生一个位于解空间的新解; 为便于后续的计算和接受, 减少算法耗时, 通常选择由当前新解经过简单地变换即可产生新解的方法, 如对构成新解的全部或部分元素进行置换、互换等, 注意到产生新解的变换方法决定了当前新解的邻域结构, 因而对冷却进度表的选取有一定的影响.

第二步是计算与新解所对应的目标函数差. 因为目标函数差仅由变换部分产生, 所以目标函数差的计算最好按增量计算. 事实表明, 对大多数应用而言, 这是计算目标函数差的最快方法.

第三步是判断新解是否被接受, 判断的依据是一个接受准则, 最常用的接受准则则是 Metropolis 准则: 若 $\Delta t' < 0$ 则接受 S' 作为新的当前解 S , 否则以概率 $\exp(-\Delta t'/T)$ 接受 S' 作为新的当前解 S .

第四步是当新解被确定接受时, 用新解代替当前解, 这只需将当前解中对应于产生新解时的变换部分予以实现, 同时修正目标函数值即可. 此时, 当前解实现了一次迭代. 可在此基础上开始下一轮试验. 而当新解被判定为舍弃时, 则在原当前解的基础上继续下一轮试验.

模拟退火算法与初始值无关, 算法求得的解与初始解状态 S (是算法迭代的起点)无关. 模拟退火算法具有渐近收敛性, 已在理论上被证明是一种以概率 1 收敛于全局最优解的全局优化算法.

模拟退火算法的应用很广泛, 可以求解 NP 完全问题, 但其参数难以控制, 其主要问题有以下三点:

- (1) 温度 T 的初始值设置问题.

温度 T 的初始值设置是影响模拟退火算法全局搜索性能的重要因素之一、初始温度高, 则搜索到全局最优解的可能性大, 但因此要花费大量的计算时间; 反之,

则可节约计算时间,但全局搜索性能可能受到影响。实际应用过程中,初始温度一般需要依据实验结果进行若干次调整。

(2) 退火速度问题。

模拟退火算法的全局搜索性能也与退火速度密切相关。一般来说,同一温度下的“充分”搜索(退火)是相当必要的,但这需要计算时间。实际应用中,要针对具体问题的性质和特征设置合理的退火平衡条件。

(3) 温度管理问题。

温度管理问题也是模拟退火算法难以处理的问题之一。实际应用中,由于必须考虑计算复杂度的切实可行性等问题,常采用如下所示的降温方式: $T(t+1) = k \times T(t)$ 式中 k 为正的略小于 1.00 的常数, t 为降温的次数。

4.5.2 改进布局算法流程

改进算法在 VPR 布局算法的基础上,采用模拟退火算法解决 FPGA 布局问题,因为模拟退火算法是个缓慢降温的过程,这里引入了精英解的概念,对搜索范围,逻辑单元交换规则等作了适当优化,用于加快模拟退火算法的速度。

算法流程如下:

(1)初始化资源,产生随机的布局作为一个初始解 S ,设置当前的最好解(精英解) $S_{best}=S$,设定一个初始温度 T ,设定每个温度下迭代的次数 $Times_Swaps_At_This_Temperature$,和搜索范围 $Range$ 。

(2)计算初始关键路径和延时矩阵,计算初始花费 C 。

(3)判断是否达到退出条件,如果没有对步骤(4),(5)循环执行 $Times_Swaps_At_This_Temperature$ 次,如达到退出条件转(7)。

(4)随机选择逻辑单元 A ,在范围 $Range$ 内随机逻辑单元 B ,然后交换这2个逻辑单元,计算花费函数的改变量 ΔC 。

(5)如果 $\Delta C < 0$,那么可以接受这个改变,否则以概率 E 接受,刚开始温度很高,大部分都可以接受,等温度逐渐降低,最后很难接受,随着温度不断降低达到算法退出条件。如果可以接受, $C=C+\Delta C$, $S=S+\Delta S$ 。

(6)降低温度,并且更新搜索范围,然后转(3)。

(7)布局结束,输出布局文件。

算法伪代码如下:

```
Alloc_FPGA_Tracks_Resources();  
Init_Logic_Block_placement();  
Init_IO_Block_placement();  
Init_Placement_Limitaion_Range();
```

```

Sbest=S=Init_Solution();
T=Init_Anneal_Temperature();
Calc_Critical_Path();
Calc_Delay_Matrix();
While !Quit_Condition()
    Repeat Times_Swaps_At_This_Temperature
        Random_Two_Logic_Block();
        Delta_S=Swap_Two_Logic_Block();
        IsAccept=Assess_Delta_S_Random();
        If(IsAccept)
            S=S+Delta_S;
            If(Cost(S)<Cost(Sbest))
                Sbest=S;
            End If
        Else
            Swap_Back_Two_Logic_Block();
        End If
    End Repeat
    Update_Temperature();
    Update_Search_Range ();
    Update_Critical_Path();
End while
Print_Placement_File();

```

4.5.3 花费函数的确定

花费函数包含2部分，网的总线长花费Wiring_Cost^[33]如式4-1和时序花费Timing_Cost如式4-6。

$$Wiring_Cost = \sum_{i=1}^{N_{nets}} q(i) \times [bb_x(i) + bb_y(i)] \quad \text{式(4-1)}$$

网的线长花费采用边界盒算法，一条网包括1个输出端(SOURCE)和若干个接收端(SINK)。bb_x(i)是网的水平跨度，即边界盒的宽度，bb_y(i)是网的竖直跨度，即边界盒的高度。如图4.6是一个有1个SOURCE和9个SINK的10端网的水平跨度和垂直跨度图。网的总线长花费是所有网的线长花费总和。

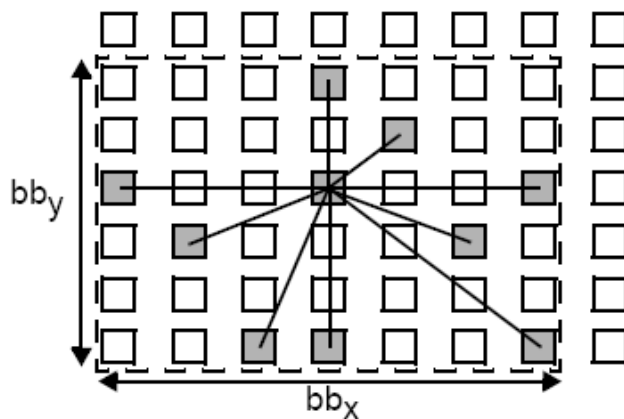


图 4.6 一个 10 端网

$q(i)$ 是 i 端网的拥挤度，这是个实验结果^[34]。如下是 1 端网到 50 端网的 q 值。

```
const float q[50] = { /* [0..49] */
```

```
1.0,1.0,1.0,1.0828,1.1536,1.2206,1.2823,1.3385,1.3991,1.4493,1.4974, 1.5455, 1.5937,
1.6418, 1.6899, 1.7304, 1.7709, 1.8114, 1.8519, 1.8924,1.9288, 1.9652, 2.0015, 2.0379,
2.0743, 2.1061, 2.1379, 2.1698, 2.2016, 2.2334,2.2646, 2.2958, 2.3271, 2.3583, 2.3895,
2.4187, 2.4479, 2.4772, 2.5064, 2.5356,2.5610, 2.5864, 2.6117, 2.6371, 2.6625, 2.6887,
2.7148, 2.7410, 2.7671, 2.7933};
```

多于 50 端的网的 q 值用式 4-2 计算。

$$q(i) = 2.79 + 0.02616 * (\text{Num_Terminals} - 50) \quad \text{式(4-2)}$$

时序花费要把电路建立一个图，IOPAD, LUT 的输入输出端口，Latch 等作为一个节点(Node)，这些节点之间的连接用边(Edge)表示。边的权值表示节点间的延时。然后从输入 IPAD 节点开始计算关键路径。

建立一个分层的时序图形，第 0 层的节点没有输入，第 n 层的节点只能是第 $(0 \dots n-1)$ 层的输出；节点之间的边是延时花费 T_{del} ；第 0 级上只有出度没有入度的节点，第 n 级上是只有扇出是 $(\geq n+1)$ 级的节点，扇入是 $(< n)$ 级的节点。

每个节点有 T_{arr} (到达时间) 和 T_{req} (过此节点要求的最迟时间) 和 $T_{del}(i, j)$ (i 节点到 j 节点的边延迟)。

故 i 到 j 的边松弛时间用式 4-3 计算：

$$\text{Slack}(i, j) = T_{req}(j) - T_{arr}(i) - T_{del}(i, j) \quad \text{式(4-3)}$$

当时序图建立起来以后，就可以计算松弛时间，以确定关键路径，首先设置第 0 层节点的到达时间为 0，然后从 $1 \dots \text{num_tnode_levels}-1$ 逐层扩散计算 T_{arr} ；再从 $\text{num_tnode_levels}-1 \dots 0$ 逐层计算 T_{req} (deadline time)；然后边松弛时间就可以计算出来，关键路径也就确定了，松弛时间为 0 的边就是关键路径上的边。

D_{max} 是最后计算出经过网络最长路径的时间，也就是经过关键路径的时间。

定义 i 到 j 的边的关键度为式 4-4，关键路径上的边的关键度为 1。

$$\text{Criticality}(i, j) = 1 - \text{Slack}(i, j) / D_{max} \quad \text{式(4-4)}$$

定义 i 到 j 的边的时序花费为式 4-5:

$$\text{Timing_Cost}(i,j)=\text{Tdel}(i,j)*\text{Criticality}(i,j) \quad \text{式(4-5)}$$

网的时序花费是所有边的时序花费之和, 如式 4-6:

$$\text{Timing_Cost} = \sum_{\forall i,j \in \text{circuit}} \text{Timing_Cost}(i,j) \quad \text{式(4-6)}$$

花费函数包含线长花费和时序花费 2 部分, 如式 4-7:

$$\text{Cost}=(1-\mu)*\text{Wiring_Cost}/\text{Previous_Wiring_Cost}+\mu*\text{Timing_Cost}/\text{Previous_Timing_Cost} \quad \text{式(4-7)}$$

μ 是时序部分所占比重, 如果 μ 为 0, 那么只考虑线长, μ 为 1, 只考虑时序。一般我们设置为 0.5, 同时考虑时序和线长, 且它们所占的比重相同, 另外考虑到 Wiring_Cost 和 Timing_Cost 的数量级不同, 在每个温度下用 Previous_Wiring_Cost 和 Previous_Timing_Cost 进行归一化处理。

4.5.4 温度更新规则

温度更新规则如下图 4.7:

Fraction of moves accepted (R_{accept})	α
$R_{\text{accept}} > 0.96$	0.5
$0.8 < R_{\text{accept}} \leq 0.96$	0.9
$0.15 < R_{\text{accept}} \leq 0.8$	0.95
$R_{\text{accept}} \leq 0.15$	0.8

图 4.7 模拟退火降温策略

R_{accept} 是在进行 Times_Swaps_At_This_Temperature 次迭代后, 成功交换的逻辑单元的比率。Tnew 是新温度, Told 是原来的温度, $T_{\text{new}}=\alpha * T_{\text{old}}$ 。

4.5.5 搜索范围更新规则

在选择 2 个逻辑单元交换的时候, 有 1 个搜索范围 Range 的限制, 刚开始的时候范围限制在整个芯片内任意交换, $\text{Range}=\min(n_x, n_y)$, n_x 和 n_y 是芯片的宽度和高度。随着退火算法的进行, 最后只能在相邻的逻辑单元间交换。经过实验发现, 最后当范围为 1 即只能在相邻的逻辑单元间交换的时候, 模拟退火算法收敛的速度太慢, 且范围为 1 阻碍了进一步对解的质量的优化。最后 Range 为 1 时占了接近 50%的运行时间, 所以这里设置为当范围为 1 的时候, $\text{Range}=\text{rand}(4)$, 即在最后的收敛阶段 Range 为 1 到 4 之间的随机数。

4.5.6 逻辑单元交换规则

如果逻辑单元交换后花费降低，则可以接受，否则以概率 E 接受。在原 VPR 布局算法中 E 为一个定值 $\exp(-\Delta C/T)$ ，在模拟退火算法的最后阶段， E 太小导致太难接受新的改变，不利于最后阶段的收敛。现对 E 作以下调整：

表 4.2 接受概率设置

Range	E
$0.3 * \min(n_x, n_y) \leq Range \leq \min(n_x, n_y)$	$\exp(-\Delta C/T)$
$0.1 * \min(n_x, n_y) \leq Range < 0.3 * \min(n_x, n_y)$	$5 * \exp(-\Delta C/T)$
$1 \leq Range < 0.1 * \min(n_x, n_y)$	$8 * \exp(-\Delta C/T)$

4.5.7 算法终止条件

算法终止条件Quit_Condition是 $T < 0.005 * Cost(S_{best}) / N_{nets}$ 。

4.6 改进的布线算法

4.6.1 采用布线资源图建立布线结构

采用布线资源图^[32]表示FPGA的布线结构，对逻辑单元CLB的SOURCE, SINK, OUTPIN, INPIN分别用 1 个布局资源节点(Routing Resource node)表示，每一条线也用 1 个布线资源节点表示，它们之间的连接用边表示。如下图 4.8 中，OUTPIN out 连接到wire3，通过一个传输开关连接wire1，然后到INPIN in1； OUTPIN out还连接到wire4，通过一个单向开关连接到wire2，然后线wire2 连接到INPIN in2。

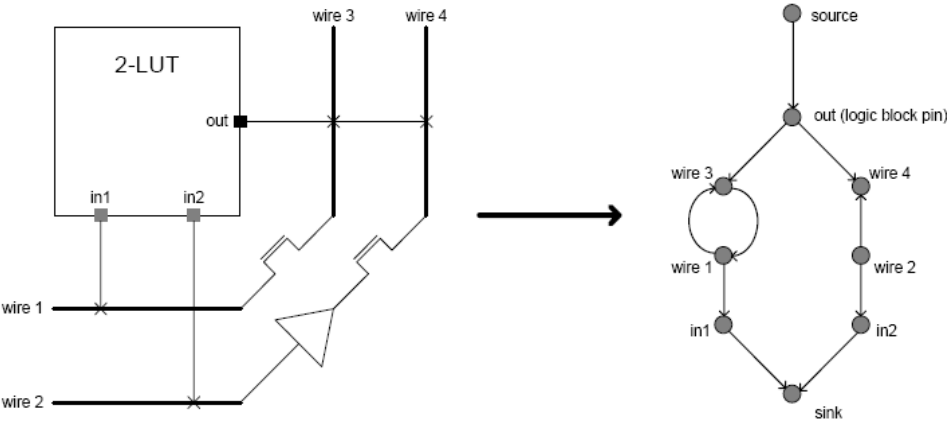


图 4.8 布线资源图

4.6.2 改进的布线算法流程

我们的改进布线算法扩展了Pathfinder Negotiated Congestion Algorithm(拥塞协

商算法)^{[1] [30]}来解决FPGA布线问题, 拥塞协商算法是对多次迭代算法的扩展。拥塞协商算法首先把所有的网布线到FPGA中, 而不考虑是否使用了过多的资源。比如某处的连接可能只能提供 3 个连接, 但是我们使用了 4 个, 然后导致拥塞的产生。当布线完成时, 会对过多使用的资源进行一个惩罚, 下次将以一个比较小的概率选择该布线资源。如果有拥塞发生, 那么擦除所有的布线, 重新进行布线, 然后再有拥塞发生, 再次惩罚, 最终经过若干次的迭代之后, 布线成功。

改进的布线算法针对原 VPR 布线算法中没有考虑内部连线的不足, 增加了一个内部连线算法, 其算法流程如下:

(1)首先记录 CLB 内部连接端口所连接的网号, 用于内部连接。

(2)建立布线通道的布线资源图。

(3)擦除所有的布线网, 对所有的网进行重新布线。对每个网布线时, 每次用 PathFinder 算法找到它的一个目标节点(SINK),然后把路径加入布线树,直到搜索完它所有的 SINK,然后更新拥塞花费。

(4)如果没有使用拥塞资源, 布线成功转(5)。否则转(3), 如果若干次迭代之后仍然无法成功布线, 布线失败, 退出。

(5)布线成功, 用(1)记录的逻辑单元内部网号进行恢复, 产生内部连接文件, 并且输出内部连接文件。

(6)输出布线文件。

算法伪代码如下:

```
Record_Sub_Block_Pin_Net_Number();
Build_IOPAD_Routing_Resource_Nodes();
Build_CLB_Routing_Resource_Nodes();
Build_TRACK_Routing_Resource_Nodes();
While IsCongestion
    Rip_Up_Routing_Nets();
    For each net Ni
        Root(RTi)=Get_SOURCE_Node(Ni);
        Num=Get_SINK_Numbers(Ni);
        Count=0;
        While Count<Num
            Sink=FindPath(RTi,Ni);
            Count=Count+1;
            Add_To_RT(RTi,Sink);
        End While
        Update_Congestion_Cost(RTi);
```

```

    End For
End While
Restore_Sub_Block_Number();
Print_Inner_Connection_File();
Print_Routing_File();

```

4.6.3 查找从源点到目标点的路径

在查找从源点 (SOURCE) 到目标点 (SINK) 的路径的时候，我们采用 PathFinder^[30]算法，PathFinder是一种考虑花费函数的启发式搜索算法。RT(Routing Tree)是网Net的布线树，已经找到了Net的部分SINK端和使用的布线资源存放在了布线树RT，现在要继续寻找Net的其他SINK节点。PathFinder算法一次找到Net的一个SINK节点，流程如下：

- (1)初始化一个优先级队列 Q。
- (2)把布线树中的每一个节点加入 Q,且优先级为 0，最高级。
- (3)从 Q 中出队一个优先级最高的节点 node，如果没有访问过，且是 SINK 节点，则找到，算法结束。
- (4)否则把 node 的所有没有访问过的邻居节点 n 以优先级 $\text{cost}(n)+\text{priority}(\text{node})$ 入队，然后转(3)。

用于布局布线的 PathFinder 算法伪代码如下：

```

FindPath(RT,Net)
  Init_Priority_Queue(Q);
  For each node in RT
    EnQueue(Q,{node,0});
  End For
While ! found
  DeQueue(Q,node);
  If !Visit(node)
    Visit(node)=TRUE;
    If Is_SINK(Net,node)
      Found=node;
      Break;
    End if
  For each neighbor n of node
    If !Visit(n)

```

```

        Queue(Q, {n, {cost(n)+priority (node)}});
    End If
End For
End If
End While

```

4.6.4 内部连线算法

为了能够得到逻辑单元内部连接信息,我们提出一个内部布线算法,该算法能够产生逻辑单元内部连接信息,为编程下载阶段提供必要的信息。主要是上面的 `Record_Sub_Block_Pin_Net_Number()` 函数和 `Restore_Sub_Block_Number()` 函数。

事实上,由于 VPR 采用了 CLB 端口等价的方法,外部布线时可以连接到与之等价的最近一个端口来减少延时,如果连接的端口发生了变化,那么其内部的连接也相应发生了变化,而 VPR 只考虑外部布线结果,没有给出内部的布线变化细节,这样就不能够反映出 CLB 内部的连接情况,而在 FPGA 设计软件的下一阶段汇编二进制流需要知道 CLB 的具体配置信息,故 CLB 内部的连接情况是需要的。

为了解决 VPR 算法中对内部互连情况缺乏这一问题,提出了一个内部连线算法,该算法可以和 VPR 中的外部连线算法一起运行,并制定一种内部连线文件格式,最终产生 FPGA 内部和外部连线文件,这 2 者后缀名分别为 `.inr` 和 `.r`,同时制作一个图形显示系统显示内部连接情况。

内部连接表示方法:

设一个 CLB 有 I 个输入端 ($0 \dots I-1$), N 个输出端 ($I \dots I+N-1$), M 个时钟 ($I+N, I+N+M-1$)。CLB 由 N 个 BLE ($0 \dots N-1$) 组成,每个 BLE 有 k 个输入端 `inputs[0...k-1]`, 1 个时钟 `clock`, 1 个输出 `output`。设当前考察的是第 m 个 BLE,记作 `BLEm` ($0 \leq m \leq N-1$)。

对 `BLEm` 的每个输入端 j ($0 \leq j \leq k-1$), 输入端 j 有三种连接情况:

- 若 CLB 输入端 i 连接到 `BLEm` 的输入端 j , 则 `BLEm` 的 `inputs[j]=i`, ($0 \leq i \leq I-1$)。
- 若 `BLEi` 输出端反馈到 `BLEm` 的输入端 j , 则 `BLEm` 的 `inputs[j]=i+I+N+M`, ($0 \leq i \leq N-1$)。
- 若没有任何连接到输入端 j , 则 `BLEm` 的 `inputs[j]=OPEN=-1`。

对 `BLEm` 的输出端有 2 种情况:

- 若 CLB 的输出端 i 连接到 `BLEm` 的输出端, 则 `BLEm` 的 `output=i`, ($I \leq i \leq I+N-1$)。
- 若 `BLEm` 的输出端未连接 CLB 任何输出端, 则 `BLEm` 的 `output=OPEN=-1`。

对BLEm的时钟端有2种情况:

- c) 若CLB的时钟端i连接到BLEm的时钟端, 则BLEm的clock=i, ($I+N \leq i \leq I+N+M-1$).
- d) 若BLEm的时钟端未连接到CLB时钟端, 则BLEm的clock=OPEN=-1。

内部布线算法的核心思想是利用内部的连接在布线过程中虽然会发生变化, 但是内部BLE端口对应的网的编号不会发生变化, 故可以在布线开始之前把内部BLE端口对应的网的编号保存下来, 然后在布线结束的时候根据布线的结果对BLE的内部连接进行恢复, 那么就可以产生内部连线的信息。因为网编号和端口编号都是用自然数表示, 为了区分, 把网编号inet变为-inet, 又因为未连接状态OPEN是-1, 为避免冲突, 最后把inet变换成-10-inet。

(1)变换, Record_Sub_Block_Pin_Net_Number()。首先读取电路文件把它转换成内部连接表示方法定义的格式。目的是把 BLE 的内部连接表示转换成带有网编号信息的形式。

假定共有X个CLB, ($0 \leq x \leq X-1$), 对每个CLBx($0 \leq x \leq X-1$)执行如下操作。

设用net[0..I+N+M-1]表示CLBx外部网的编号。对每个BLEm($0 \leq m \leq N-1$)的inputs[0...k-1], output, clock的取值情况作下面变换, 共有下面4种情况:

- a) 值为i, ($0 \leq i \leq I+N+M-1$), 若net[i]为OPEN, 这是默认直接Hook的情况, 那么不进行变换。
- b) 值为i, ($0 \leq i \leq I+N+M-1$), 若net[i]不为OPEN, 那么把网的编号记录到内部信息, 若是BLEm的j输入端, 则inputs[j]=-10-net[i]; 若是BLEm的输出端, 则output=-10-net[i]; 若是BLEm的时钟端, 则clock=-10-net[i]。
- c) 值为OPEN, 说明没有连接, 则仍然为OPEN。
- d) 值大于I+N+M-1, 这是内部BLE输出端到内部BLE输入端反馈情况, 没有连接到CLB外部端口, 不进行变换。

变换算法的时间复杂度分析: 对所有使用的BLE做变换, 最多有XN个BLE, 每个BLE最多使用K+2个端口。所以时间复杂度 $O(XN(K+2))$ 。一般X为几百到几千, N为4到10, K为4到6。变换操作在布线过程中只需要执行一次, 这个时间对于布线算法是微不足道的, 并且没有使用额外的存储空间。

(2)恢复, Restore_Sub_Block_Number。设外部布线结果共有T条布线网络, 对每个布线网络($0 \leq t \leq T-1$), 检查它连接到CLB的输入端, 输出端和时钟端。

- a) 若网络t连接到CLBx的输入端i, 检查CLBx的所有BLEm($0 \leq m \leq N-1$), 对所有满足 $-(10+BLEm.inputs[j])$ 和t相等的BLEm输入端j, 令BLEm.inputs[j]=i。
- b) 若网络t连接到CLBx的时钟端i, 检查CLBx的所有BLEm($0 \leq m \leq N-1$), 如能找到BLEm时钟端使得 $-(10+BLEm.clock)$ 和t相等, 则BLEm.clock=i。
- c) 若网络t连接到CLBx的输出端i, 检查CLBx的所有BLEm($0 \leq m \leq N-1$), 如

能找到BLEm输出端使得 $-(10+BLEm.output)$ 和 t 相等, 则 $BLEm.output=i$ 。

恢复算法的时间复杂度分析: 对所有布线网, 假如一个布线网最多有 S 个端口, 那么最多有 TS 个端口需要检查, 检查每个端口最多需要比较 $N(K+2)$ 次, 所以时间复杂度为 $O(TSN(K+2))$, 在原结构上的修改, 没有使用额外的存储空间。

4.7 实验方法与实验结果

4.7.1 实验方法

(1) 布局布线与FPGA芯片相关, 故首先采用第三章的结构描述方法对1个FPGA 建立抽象模型, 生成 1个结构描述文件;

(2) 划分网表工具转换工艺映射后的网表文件(从布局布线的前一阶段得到)生成逻辑单元网表和LUT配置文件;

(3) 混合布局布线算法读取逻辑单元网表和结构描述文件, 生成布局文件, 布线文件和逻辑单元内部连接文件;

(4) 前面产生的4个文件: LUT配置文件, 布局文件, 布线文件和逻辑单元内部连接文件送入FPGA编程工具(布局布线的下一阶段)生成位流文件用于下载。

4.7.2 实验结果

改进的布局布线算法增强了对 FPGA 结构的支持, 主要考虑如何使模拟退火算法和拥塞迭代布线算法用于实际的 FPGA 芯片, 支持更丰富的结构等, 改进算法能够生成布局文件, 布线文件和内部连接文件用于编程下载。

如图4.9是开发的FPGA布局布线工具对16位全加器电路的布局布线结果, 每个CLB内部含有4个BLE, 未使用的BLE没有画出。CLB的内部连线包括BLE与BLE之间, CLB与BLE之间的连线。外部连线是CLB与布线通道(track)之间, IOPAD与布线通道之间, 布线通道与布线通道之间的连线。

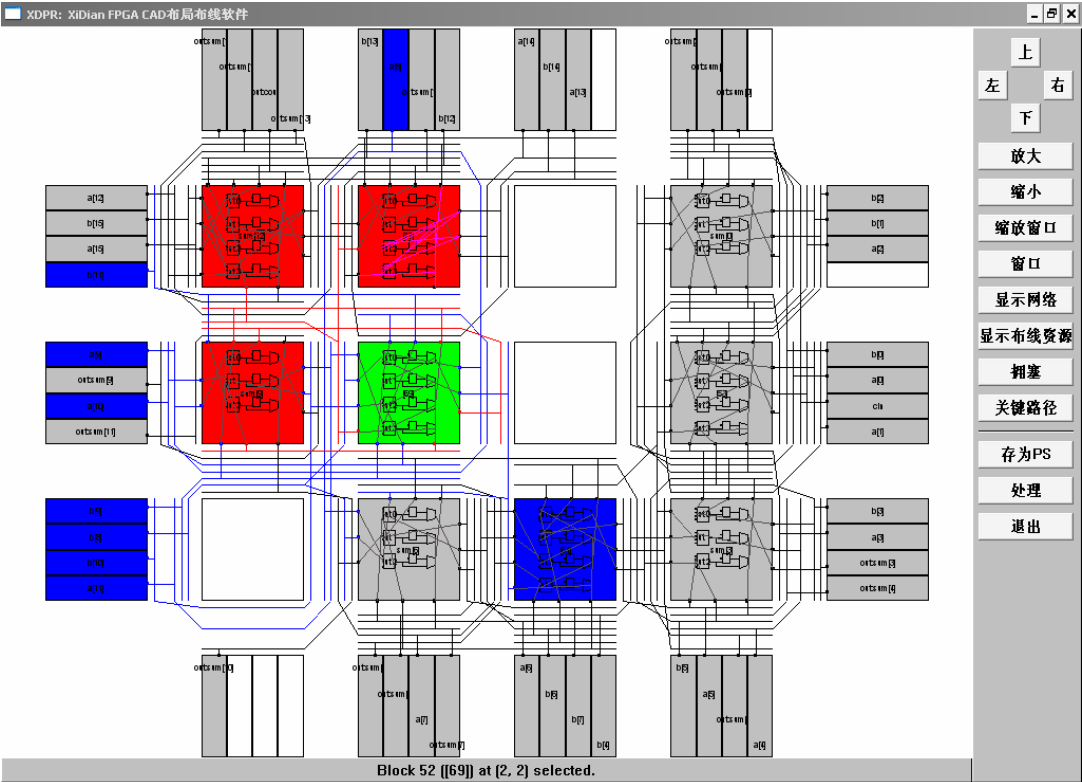


图 4.9 包含内部连线的布局布线结果图

VPR和一些对VPR的改进算法^[31]都是使用VPR提供的 20 个最大的MCNC^[35]标准电路来进行算法好坏的测试。这里也采用这 20 个标准电路，在相同的FPGA结构(使用含 4 个BLE 的 CLB)上进行实验，表 4.3 和表 4.4 是改进算法MPR和VPR算法的布局结果对比，表 4.5 是改进算法MPR和VPR算法的布线结果对比。

表 4.3 VPR 和 MPR 布局结果比较

电路名 称	输入	输出	含有 CLBs	FPGA SIZE	布局逻辑单元交换次数		
					VPR	MPR	Radio= (MPR-VPR)/VPR *100%
ex5p	8	63	278	17×17	2481166	1860072	-25. 03%
misex3	14	14	361	19×19	2782220	2126710	-23. 56%
apex4	9	19	335	19×19	2537122	1807677	-28. 75%
alu4	14	8	390	20×20	3003700	2250450	-25. 07%
tseng	52	122	266	17×17	3847785	3180118	-17. 35%
seq	41	35	448	22×22	4196892	2881364	-31. 34%
apex2	38	3	485	23×23	4360198	3475296	-20. 29%
diffeq	64	39	379	20×20	3967215	2853866	-28. 06%
dsip	229	197	343	27×27	8875062	6798734	-23. 39%
des	256	245	415	32×32	10316924	8838473	-14. 33%
s298	4	6	489	23×23	4075813	3377958	-17. 12%
bigkey	229	197	427	27×27	10352384	8352384	-19. 31%
spla	16	46	955	31×31	10633896	8823932	-17. 02%
frisc	20	116	893	30×30	11424710	9604453	-15. 93%
elliptic	131	114	906	31×31	13265450	10818723	-18. 44%
pdc	16	40	1187	35×35	14430420	10632975	-26. 31%
ex1010	10	10	1200	35×35	13684545	10238834	-25. 17%

s38584	38	304	1614	41×41	27145938	22034843	-18.83%
s38417	29	106	1610	41×41	22473424	17693287	-21.27%
clma	62	82	2137	47×47	29837421	23656670	-20.71%
平均值							-21.86%

表 4.4 VPR 和 MPR 布局花费结果比较

电路名	布局线长花费			布局时序花费 (e-6)		
	VPR	MPR	Radio= (MPR-VPR)/VPR *100%	VPR	MPR	Radio= (MPR-VPR)/VPR *100%
ex5p	106.696	107.523	0.77%	2.16829	2.05734	-5.11%
misex3	123.679	122.331	-1.08%	2.24264	2.12487	-5.25%
apex4	122.489	123.871	1.12%	2.92818	3.09599	5.73%
alu4	124.159	123.919	-0.19%	3.31893	3.28629	-0.98%
tseng	69.401	70.293	1.28%	2.39310	2.41237	0.80%
seq	167.601	167.904	0.18%	3.14704	2.88233	-8.41%
apex2	186.024	185.637	-0.21%	3.32117	3.19351	-3.84%
diffreq	100.972	101.017	0.04%	2.90151	2.93130	1.02%
dsip	117.085	117.642	0.47%	0.91594	0.93567	2.15%
des	159.956	160.343	0.24%	1.11377	1.01634	-8.74%
s298	121.765	124.406	2.16%	2.71827	2.56296	-5.71%
bigkey	131.934	131.232	-0.53%	2.89744	2.83450	-2.17%
spla	427.085	426.515	-0.13%	2.82423	2.81542	-0.31%
frisc	351.745	349.434	-0.65%	1.00839	1.10923	10.00%
elliptic	341.859	342.321	0.14%	1.68419	1.69451	0.61%
pdc	610.444	608.238	-0.36%	6.69785	6.34983	-5.19%
ex1010	467.804	468.948	0.24%	3.58151	3.66766	2.41%
s38584	528.958	529.563	0.11%	5.46175	5.29934	-2.97%
s38417	547.514	546.756	-0.14%	3.39473	3.48532	2.66%
clma	632.284	634.323	0.32%	4.39949	4.25884	-3.19%
平均值			+0.19%			-1.32%

表 4.5 VPR 和 MPR 布线结果比较

电路名	布线所需最小通道宽度			布线关键路径延时(e-8s)		
	VPR	MPR	差值= (MPR-VPR)/VPR *100%	VPR	MPR	Radio= (MPR-VPR)/VPR *100%
ex5p	42	41	-2.38%	4.72653	4.48800	-5.04%
misex3	37	38	2.70%	4.54200	4.69485	3.36%
apex4	39	41	5.12%	5.29409	4.91968	-7.07%
alu4	33	34	3.03%	4.59849	4.56774	-0.66%
tseng	24	24	0%	4.70478	4.81237	2.28%
seq	41	39	-4.87%	6.04919	6.14807	1.63%
apex2	43	43	0%	5.01283	5.04335	0.61%
diffreq	29	27	-6.89%	4.74791	4.81005	1.30%
dsip	24	24	0%	3.02915	3.01605	-0.43%
des	25	25	0%	4.80645	4.70613	-2.08%
s298	31	28	-9.67%	6.34228	6.10573	-3.72%

bigkey	23	23	0%	8.23312	7.81514	-5.07%
spla	55	54	-1.81%	6.39316	6.10248	-4.54%
frisc	45	46	2.22%	7.84923	7.43566	-5.26%
elliptic	41	42	2.43%	6.84939	6.91847	1.01%
pdc	60	58	-3.33%	7.49437	7.19930	-3.93%
ex1010	45	44	-2.22%	8.40430	7.92038	-5.75%
s38584	33	34	3.03%	7.74845	7.91937	2.21%
s38417	37	38	2.70%	8.43214	8.23923	-2.28%
clma	46	46	0%	8.39983	8.45784	0.69%
平均值			-0.49%			-1.64%

布局的逻辑单元交换是布局中最耗时间的操作, 模拟退火布局算法通过不断地交换逻辑单元然后评价是否可以接受, 来改进布局质量, 逻辑单元交换次数反映了布局时间的长短, 交换次数越少, 布局速度越快。布局花费反映了布局之后解的质量, 布局花费越少, 布局后的质量越高。从表 4.3 中可以看出改进算法 MPR 比 VPR 大概减少了 21%逻辑单元交换次数, 在表 4.4 中布局的线长花费增加 0.19%, 布局的时序花费减少 1.32%。

布线所需最小通道宽度反映了布线算法的可布通性, 算法所需要的布线通道宽度越小, 具有更好的布通性, 能够在具有较少布线资源的情况下, 更容易布通。布线关键路径延时反映了布线结果的质量, 延时越少, 质量越高。布局的结果要为布线服务, 从表 4.5 最后的布线结果来看, 在可布通性方面跟 VPR 差不多, 减少了 0.49%的布线通道需求, 关键路径延时减少 1.64%。

从比较结果上可以看出, 改进的布局布线算法在对相同结构的 FPGA 布局布线时, 最后解的质量上没有低于 VPR, 而在功能上支持更多的 FPGA 结构特性(开关盒矩阵, 互连线的开始点, 互连线连接盒分布, 互连线开关盒分布等), 支持内部连线。在布局的速度上减少了 21%的逻辑单元交换次数。

VPR 是个抽象的布局布线工具, 没有考虑如何把布局布线算法应用到实际的芯片用于编程下载, 与 VPR 相比, 当前开发的布局布线工具支持更多的 FPGA 结构特征, 改进的布局布线算法生成了内部连接文件, 布局文件, 布线文件, 这 3 个文件和 LUT 配置文件可以提供给编程下载阶段, 用于生成位流文件下载到 FPGA 芯片中。

4.8 本章小结

本章讨论了划分网表工具, 布局布线算法。改进的划分网表工具是从 VPACK 中修改, 增加了生成 LUT 配置文件的功能。改进的布局布线算法与 VPR 相比, 更多的考虑了如何把算法应用于实际的 FPGA 芯片, 支持更多的 FPGA 结构特征, 为了支持编程下载, 还产生了布局文件, 布线文件, 内部连线文件。LUT 配置文件, 布局文件, 布线文件, 内部连接配置文件提供给布局布线的下一阶段(编程下

载)生成位流文件, 然后下载位流文件到 FPGA 中。

当前已经完成的布局布线软件对 FPGA 结构特性支持如表 4.6:

表 4.6 目前开发的布局布线工具对 FPGA 结构的支持

FPGA结构特征	是否支持
芯片大小	√
X,Y通道宽度	√
IO通道宽度	√
CLB到通道连接情况	√
IOPAD到通道连接情况	√
CLB大小描述	×
Subset开关盒样式	√
指定的开关盒样式	×
面积模型	√
延时模型	√
混合逻辑单元	×
线的模型	√
开关盒的分布	√
连接盒的分布	√
逻辑等价的端口	√
开关样式	√
指定IOPAD	√
不同的通道不同的连接	×

第五章 FPGA 支持软件功能验证

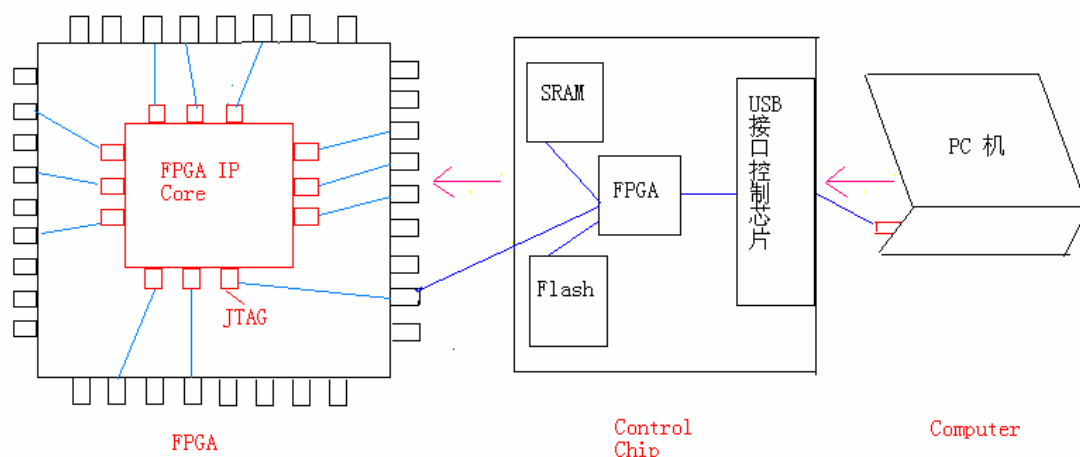


图 5.1 FPGA 支持软件验证图

目前我们的 FPGA 支持软件已经完成了设计输入，逻辑综合，工艺映射，布局布线，编程，生成了最后的位流文件，可以下载到芯片中去。但是相关的硬件 FPGA 芯片还没有设计及流片，且流片费用比较高，为了验证我们 FPGA 支持软件的功能正确性，这里提出了一种可以验证 FPGA 支持软件正确性的方法如图 5.1。

要验证 FPGA 支持软件，我们需要有个芯片能够提供给我们 FPGA 芯片的功能，商业的 FPGA 的结构和具体的参数我们不知道，且编程下载接口没有开放。即使自主知识产权的 FPGA 设计了出来，但是必须流片才能进行 FPGA 支持软件的验证。这里采取用 FPGA 软核来代替 FPGA 进行测试，而且这个软核只需要用 RTL 级的硬件描述语言描述 FPGA 的逻辑功能就可以了，比实际设计 FPGA 芯片要简单得多，不用考虑版图，后端设计，金属层连接等，因为我们只关心 FPGA 支持软件功能是否正确，看我们最后下载到软核中的位流能否真正起作用。采用 FPGA 软核的优势：

- a) 设计简单，不用考虑后端设计，只需实现逻辑功能。
- b) 成本低，不需流片。
- c) 结构改变灵活，可以测试 FPGA 软件支持的不同的结构。

5.1 FPGA 软核设计

应主要对 FPGA 以下 3 种元素建立模型：

(1) 逻辑单元内部功能，比如用 verilog 对基本逻辑单元 BLE 建立模型如下，BLE 由一个 LUT 和一个 D flip-flop 组成。

```
module LUT(out,sram,sel);
```

```
input [15:0]sram;
input[3:0] sel;
output out;
reg out;
always @(sel)
begin
out=sram[sel];
end
endmodule

module DFF(q,d,clk);
input d,clk;;
output q;
reg q;
always @(posedge clk)
begin
q <= d;
end
endmodule

module BLE(sel,clk,out)
input sel,clk;
output out;
reg out;
reg [15:0]sram;
reg sram_out;
LUT lut1(sram_out,sram,sel);
DFF dff1(out,sram_out,clk);
Endmodule
```

(2)逻辑单元 CLB 输入输出端口和 IOPAD 与线的连接,可以考虑采用多路选择器或者 3 态传输门实现,线可以连接逻辑单元 CLB 的输入输出端口,或者 IOPAD。3 态传输门使能的时候,建立端口和线的连接,或者 CLB 的输入端口因为只能连接到一条线,可以用多路选择器来决定它连到哪条线。

(3)线与线连接的开关矩阵,可以考虑用 3 态传输门实现。3 态传输门使能的时候建立线与线的连接。

另外还要建立 JTAG 控制电路,来控制内部 LUT 的配置,开关矩阵(线与线的 3 态使能)的配置,连接盒的配置等,这部分控制电路用来完成位流的下载。

5.2 控制芯片设计

为了把位流下载到 FPGA 软核中,还必须一个接口控制芯片,Altera 公司下载就需要一个专用的下载电缆。我们这里这个芯片负责跟 PC 机通信,PC 机发送控制命令,然后控制芯片中的 FPGA 通过 USB 接口控制芯片,接收到数据和命令后进行处理,通过 FPGA 软核开放的 JTAG 接口进行下载。也可以用其它接口和 PC

机通信，比如 PCI。

USB 接口控制芯片：负责跟 PC 机进行通信；接收 PC 机的命令和数据，传给控制芯片中的 FPGA。

控制芯片中的 FPGA：负责收到 PC 机的命令和数据，然后进行解释，处理，转换成 JTAG 可以下载的命令字等，控制下载整个过程。还可以执行一些附加操作，比如传递给 PC 机当前下载进度等数据。

SRAM：为了加快速度，控制芯片中的 FPGA 有可能要接收大量下载数据，存放在这里。

Flash：可以把一些配置数据预先放在里面，也可以存放一些动态布局布线的操作命令。控制芯片中的 FPGA 加载这些命令然后执行。

5.3 PC 机程序设计

需要设计 2 个程序：1 个 PC 机接口驱动程序，1 个 PC 机下载程序。

接口驱动程序：如果使用的是 USB 接口，需要写相应的 USB 接口驱动程序，其他的接口也一样。主要是因为通过接口我们需要发送一些特殊的命令来控制控制芯片的 FPGA 运行，传输数据等，这些需要以一个双方都了解的格式告诉 FPGA。

PC 机下载程序：用来把位流文件下载到 FPGA 软核中，我们可以定义一组 API 函数，API 函数调用的是驱动中的函数，API 函数的功能可以有：设置某个 LUT 配置的 API，设置某个开关矩阵的 API 等。这组 API 和驱动程序可以形成一个自定义的标准，当软核的结构或 JTAG 接口改变的时候，只需修改控制芯片中的 FPGA 控制程序，驱动程序和 API 不变。PC 机下载程序就是调用这些 API 函数来实现下载功能。

5.4 本章小结

本章讨论了如何对我们的 FPGA 支持软件进行功能验证，提出一个可以进行编程下载到 FPGA 软核然后进行功能验证的方案。当 FPGA 实际流片之后，我们设计的控制芯片，驱动程序，和 PC 机软件经过少量修改还可以用于实际的 FPGA 芯片的下载。

第六章 结论与展望

6.1 结论

本文主要介绍了 FPGA 的结构, 和 FPGA 支持软件设计流程中的划分算法, 布局算法, 布线算法, 主要完成了以下 3 方面的工作:

(1)用 FPGA 结构描述方法对 FPGA 建立模型

本文设计了 FPGA 结构描述方法, 可以用来对 FPGA 建立抽象模型, 用于布局布线软件, 当要对不同的 FPGA 进行布局布线时, 仅需要修改结构描述文件, 而不需要修改布局布线算法。主要是对 FPGA 的 10 个关键特性进行了抽象, 基本包括了 FPGA 结构的方方面面。

(2)基本完成一个划分网表工具和一个布局布线工具, 可以配合编程下载

改进的划分网表工具可以生成 LUT 配置信息文件, 改进的布局布线算法能够支持更多的 FPGA 结构特征。开发的布局布线工具在可布通性上和 VPR 接近, 布局阶段能够减少 21%的逻辑单元交换次数, 它在布局布线之后生成内部连接信息文件, 布局文件和布线文件。这些文件提供给布局布线的下一阶段编程下载必要的信息, 可以生成位流文件下载到 FPGA 中。

(3)可采用 FPGA 软核的方法进行 FPGA 功能验证

目前我们的 FPGA 支持软件已经基本完成, 但是还没有相应的 FPGA 芯片来把位流文件下载到芯片里面来完成最后的软件验证。我们可以采用 FPGA 软核的方法来验证我们的 FPGA 支持软件的功能正确性。

6.2 展望

(1)完善布局布线工具。我们完成的布局布线工具在功能上虽然已经支持 FPGA 结构的各种特征, 不足之处是还仅支持单一的逻辑单元, 下一步要扩充布局布线工具的功能, 支持不同的逻辑单元如 RAM 等, 从而更加接近商业的 FPGA 结构。

(2)用提出的验证方案验证支持软件。目前的我们布局布线工具虽然对 FPGA 的结构的支持还不完美, 但是已经完成了布局布线的各种功能, 且能够配合编程下载阶段生成位流文件, 下一步需要用我们提出的验证方案对 FPGA 支持软件进行功能上的验证, 为了适应不同的芯片结构, 就需要开发一系列的可扩展的验证程序。

致 谢

毕业论文的顺利完成，凝聚着许多人的关怀和帮助。

首先要感谢我敬爱的段振华教授在学术上对我的精心指导和严格要求，在课题方面给予我及时的指导和帮助。学到的专业知识和做人的道理将使我终身受益，并激励着我在今后的人生道路上不断开拓进取。在此，我再一次对段老师的培养和关怀表示诚挚的谢意！

非常感谢我的父母对我的无微不至的关怀和帮助，对他们的关怀和帮助致以诚挚的谢意！

同时非常感谢研究所鱼滨，祁建军等老师的指导和督促，在与他们的讨论交流中，开拓了我的思路，我对他们致以真诚的谢意！

非常感谢我们组其他同学，王晓光，吕晋鹏，范全润，王石，赵峥，孙虎，肖艳等同学，我们聚在一起经常讨论问题，他们兢兢业业、对工作认真负责的态度为我做出了好的表率，时刻激励我努力向前，在此对他们表示感谢。

另外，也感谢其他组对我工作的支持。

同时感谢宋永波同学，马永涛同学对我的支持与帮助。

最后，向所有帮助过我的人致以最诚挚的谢意。

参考文献

- [1]V.Betz, J.Rose,and A.Marquardt."Architecture and CAD for Deep-Submicron FPGAs". Kluwer Academic Publishers, New York, 1999.
- [2]Jianshe He, "Technology Mapping and Architecture of Heterogenous Field-Programmable Gate Arrays," M.A.Sc. Thesis, University of Toronto, 1993.
- [3]Xilinx Databook, 1996.
- [4]Altera Databook, 1996.
- [5]He J,Rose J,"Advantages of heterogeneous logic block architecture for FPGAs",Custom Integrated Circuits Conference, 1993., Proceedings of the IEEE 1993.
- [6]Vaughn Betz , Jonathan Rose,"Automatic Generation of FPGA Routing Architectures from High-Level Descriptions",ACM/SIGDA International Symposium on Field Programmable Gate Array,2000.
- [7]Jordan S.Swartz,"A High-Speed Timing-Aware Router for FPGAs",Master of Applied Science,Department of Electrical and Computer Engineering University of Toronto,1998.
- [8]J. Cong, J. Peck, and Y. Ding. "RASP: A General Logic Synthesis System for SRAM-based FPGAs". In ACM Symp. on FPGAs, pages 137–143, 1996.
- [9]Breuer ,M.A.1977,"Min-cut placement." Journal of Design Automation and Fault Tolerant Computing, Vol.1,no.4,pp.343-362.
- [10]Kirkpatrick,S.,C.D.Gerlatt Jr.,and M.P.Vecchi.1983."Optimization by simulated annealing",Science,Vol.220,no.4598,pp.671-680.
- [11]Sechen,C.1988.VLSI Placement and Global Routing Using Simulated Annealing.Norwell,MA:Kluwer.ISBN 0898382815.TK7874.S38.
- [12]Vaughn Betz and Jonathan Rose,"VPR:A New Packing,Placement and Routing Tool for FPGA Research",International Workshop on Field Programmable Logic and Applications, 1997.
- [13]Alexander(Sandy)Marquardt,Vaughn Betz and Jonathan Rose, "Timing-Driven Placement for FPGAs",ACMISIGDA International Symposium on Field Programmable Gate Arrays. 2000.
- [14]Pritha Banerjee,"Accelerators for FPGA Placement",The 4th Annual Inter Research Institute Student Seminar in Computer Science Indian Institute of Technology,2005.
- [15]Pongstorn Maidee, Cristinel Ababei, Kia Bazargan, Member, IEEE "Timing-driven Partitioning-based Placement for Island Style FPGAs",IEEE

Transactions on Computer-Aided Design of Integrated Circuits and Systems, Vol. 24, No. 3, Mar. 2005

[16]Malay Haldar,Anshuman Nayak,Alok Choudhary and Prith Banerjee,"Parallel Algorithms For FPGA Placement",Great Lakes Symposium on VLSI,Proceedings of the 10th Great Lakes symposium on VLSI,2000.

[17]Pak K.Chan and Martine D.F.Schlag,"Parallel Placement for Field Programmable Gate Arrays", FPGA'03,February 23-25,2003,Monterey,California,USA.

[18]C.Y.Lee,"An Algorithm for Path Connections and its Applications," IRE Trans.Electron.Comput.,Vol. EC=10,1961, pp.346-365.

[19]F.Rubin,"The Lee Path Connection Algorithm," IEEE Trans. Computers, Sept.1974, pp.907 -914.

[20]J. Soukup, "Fast Maze Router," Proc. 15th Design Automation Conf., June 1978, pp. 100-102.

[21]R.Linsker,"An iterative-improvement penalty-function driven wire routing system," IBM J. Res. Develop., vol. 28, no. 5, 1984, pp. 613-624.

[22]E.S.Kuh and M. Marek-Sadowska, "Global routing," in Layout Design and Verification, T. Ohtsuki, Ed. New York: Elsevier Science, 1986, pp. 169-198.

[23]R.Nair,"A Simple Yet Effective Technique for Global Wiring," IEEE Trans. on CAD, March 1987, pp. 165-172.

[24]S.Brown, J.Rose, Z. G. Vranesic, "A Detailed Router for Field-Programmable Gate Arrays," IEEE Trans. on CAD, May 1992, pp. 620 - 628.

[25]S.Brown, M. Khellah and G. Lemieux, "Segmented Routing for Speed-Performance and Routability in Field-Programmable Gate Arrays," Journal of VLSI Design, Vol. 4, No. 4, 1996, pp. 275 - 291.

[26]G.Lemieux, S.Brown, "A Detailed Router for Allocating Wire Segments in FPGAs," ACM/SIGDA Physical Design Workshop, 1993, pp. 215 - 226.

[27]M.Alexander,J.Cohoon,J.Ganley and G. Robins, "Performance-Oriented Placement and Routing for Field-Programmable Gate Arrays," European Design Automation Conf., 1995, pp. 80 - 85.

[28]Y.S.Lee,A.Wu,"A Performance and Routability Driven Router for FPGAs Considering Path Delays," DAC, 1995, pp. 557 - 561.

[29]Y.-L.Wu, M.Marek-Sadowska, "An Efficient Router for 2-D Field-Programmable Gate Arrays," European Design Automation Conf., 1994, pp. 412 - 416.

[30]C.Ebeling, L.McMurchie, S.A.Hauck and S.Burns, "Placement and Routing Tools for the Triptych FPGA," IEEE Trans. on VLSI, Dec. 1995, pp. 473 - 482.

-
- [31]Roman Lyseckf, Frank Vahida, Sheldon X.D.Tanb, "Dynamic FPGA Routing for Just-in-Time FPGA Compilation", Design Automation Conference, 2005
- [32]Ajay Roopchansingh, "Nearest Neighbour Interconnect Architecture In DEEP SUBMICRON FPGAS", Ms.D.Thesis, University of Toronto, 2002.
- [33]V.Betz, J.Rose, and A.Marquardt, Architecture and CAD for Deep-Submicron FPGAs, Kluwer Academic Publishers, February 1999.
- [34]C.Cheng, "RISA: Accurate and Efficient Placement Routability Modeling," ICCAD, 1994, pp. 690 - 695.
- [35]S.Yang, "Logic Synthesis and Optimization Benchmarks User Guide Version 3.0", MCNC International Workshop on Logic Synthesis, 1991.

作者在读期间研究成果

在硕士期间取得的研究成果如下：

一、参加科研情况

[1]参加科研项目“FPGA 支持软件技术研究”，研究对 FPGA 芯片结构建模，布局布线算法等，在 linux(Fedora 4)平台用 c/c++语言开发了单元网表划分工具和 FPGA 布局布线工具。

二、发表论文情况

[1]赵刚。“基于 VPR 的 FPGA 内部布线算法”，2007 年西安电子科技大学研究生学术年会。



西安电子科技大学

地址：西安市太白南路2号

邮编：710071

网址：www.xidian.edu.cn